ANEP · DIRECCIÓN GENERAL DE EDUCACIÓN TÉCNICO PROFESIONAL

UTU · ESCUELA TÉCNICA DE PANDO · ANEP

TGS — Taller Global de Servicios

Plataforma web de gestión de servicios mecánicos para clientes, gestores de taller y administración

Documento técnico del Proyecto de Pasaje de Grado

Santiago Figueroa · C.I. 5.777.974-1

Alex Torres · C.I. 6.747.915-7

Facundo Fernandez · C.I. 5.717.364-8

Mariano Di Trani · C.I. 5.743.443-6

Alejo Mussetti · C.I. 5.750.437-6

Asignatura: Tecnologías de la Información (BT) — Proyecto de Pasaje de Grado

Curso: 3.º de Mediaón — 3.º de Tecnología de la Información (3ME – 3MD)

Empresa cliente simulada: TGS — Blue Tech Uruguay

Tipo de entrega: Documento técnico y memoria del proyecto (2.º y 3.º entrega)

Fecha de entrega: octubre de 2026

Ciudad: Canelones, Uruguay

Repositorio: github.com/ttugol909674-cpu/index

Contenido

1	Resumen	3
2	Introducción: contexto, justificación, objetivos, alcance y marco regulatorio	4
3	Marco teórico y metodología	7
4	Requisitos del sistema y matriz de trazabilidad	10
5	Diseño del sistema: los seis diagramas UML	17
6	Base de datos: MER, diccionario de datos y normalización	23
7	Implementación del sistema (con capturas de las 24 pantallas)	31
8	Seguridad de la información	48
9	Plan y resultados de las pruebas	50
10	Trabajo en equipo y control de versiones	53
11	Infraestructura y administración de sistemas operativos	55
12	Glosario bilingüe	60
13	Conclusiones y trabajo futuro	63
14	Referencias	64
A	Anexos: credenciales, estructura de archivos y checklist normativo	65

LAS FIGURAS DEL CAPÍTULO 7 MUESTRAN EL SISTEMA REAL EN FUNCIONAMIENTO EN LA MÁQUINA VIRTUAL DEL PROYECTO.

1. Resumen

Este documento presenta el desarrollo completo del proyecto **TGS — Taller Global de Servicios**, una plataforma web que permite a los clientes de una red de talleres mecánicos registrar sus vehículos, agendar turnos de servicio, consultar el historial de mantenimiento, pagar los servicios y descargar sus facturas; al gestor de taller le permite gestionar la agenda del día, validar los pagos reportados y emitir presupuestos; y al administrador le permite administrar usuarios, sedes, servicios, la matriz de permisos y los datos bancarios de la empresa.

El sistema no es una maqueta: es una aplicación completa que funciona de punta a punta sobre una máquina virtual Ubuntu 26.04 con Docker Compose, dos contenedores (Apache + PHP 8.4 y MySQL 8.4), un volumen persistente para la base, scripts de instalación, respaldo, restauración y simulación de caída, y una suite automatizada de **28 pruebas end-to-end más 20 casos de verificación de requisitos**, todos en verde. La base de datos tiene **16 tablas** normalizadas hasta 3FN, con datos de prueba cargados por script y una migración idempotente que permite evolucionar el esquema sin perder información.

La seguridad se resolvió con consultas preparadas (PDO) en el 100 % de las operaciones, cifrado de contraseñas con bcrypt, token CSRF en todos los formularios POST, matriz de permisos por rol editable en caliente y gestión de errores que nunca muestra detalles al visitante. La interfaz es bilingüe (es/en) y responde a tres perfiles de uso distintos, con 24 pantallas documentadas en este documento mediante capturas reales del sistema en funcionamiento.

PALABRAS CLAVE

PHP 8.4 · MySQL 8.4 · Docker · PDO · CSRF · bcrypt · UML · Scrum · Web app · Servicios mecánicos

2. Introducción

2.1 Contexto y problemática

Las redes de talleres mecánicos suelen resolver la gestión de turnos por cuaderno, por WhatsApp o por llamadas telefónica. Eso genera tres problemas habituales: el cliente no sabe con certeza si su vehículo está siendo atendido ni cuándo estará listo; el taller no sabe con anticipación cuántos lugares tiene disponibles para el día siguiente y termina aceptando reservas que no puede cumplir; y, cuando el trabajo termina, la facturación y el cobro quedan en un papel que se pierde o se transcribe a mano con errores.

La empresa cliente del proyecto, **TGS — Blue Tech Uruguay**, planteó la necesidad de una plataforma única que ordene el ciclo completo del servicio: agendar, atender, presupuestar, cobrar y documentar. El enunciado oficial (ANEP–DGETP, *Proyecto de Pasaje de Grado*) precisa el alcance: una página web y una aplicación integrada que permita registrarse, agendar servicios, mostrar un mapa con los talleres y su ubicación, considerar la disponibilidad de lugar por sede, realizar reservas y efectuar pagos, con herramientas de gestión para la administración de TGS.

2.2 Justificación del proyecto

El proyecto se justifica en cuatro planos:

- **Funcional:** cubre los 40 requisitos funcionales del enunciado, organizados en tres roles (cliente, gestor de puntos de carga y administrador) tal como fueron especificados.
- **Técnico:** la aplicación se despliega de forma reproducible en cualquier equipo con Docker, con la configuración externalizada y un juego de scripts que permiten instalarla, respaldarla, restaurarla y simular una caída del servidor durante la defensa.
- **Académico:** la solución aplica de forma explícita los contenidos de Programación Full Stack (base de datos, backend, conexión), Ingeniería de Software (UML, lógica del sistema, diseño de interfaz, control de versiones, plan de pruebas) y Administración de Sistemas Operativos (servidor, respaldos, seguridad, automatización).
- **Social:** al digitalizar la reserva y el cobro se reduce la barrera de entrada para personas con menos familiaridad con la informática y se mejora la включіón digital de los clientes mayores o con movilidad reducida, que pueden agendar y pagar desde el celular.

2.3 Objetivo general y objetivos específicos

El **objetivo general** del proyecto es desarrollar una solución tecnológica que cumpla los requisitos planteados por la empresa mediante la creación de una página web y una aplicación integrada, capaz de ofrecer a los usuarios registrarse, reservar servicios, planificar los mantenimientos considerando los talleres disponibles y realizar pagos de manera segura.

Los **objetivos específicos** tomados del enunciado se operationalizaron así:

1. Diseñar una base de datos para almacenar la información del negocio (modelo normalizado 3FN, 16 tablas, DDL y datos de prueba en scripts versionados).

2. Implementar un servidor web reproducible (Apache + PHP 8.4 en Docker sobre Ubuntu, con MySQL 8.4 como gestor de base de datos).
3. Crear una interfaz de usuario atractiva, intuitiva y fácil de utilizar (diseño unificado en tres paneles, navegación coherente, validación en cliente y servidor, mensajes de error claros).
4. Implementar un sistema de inicio de sesión que permita a los administradores gestionar la información (sesiones con cookies protegidas, regeneración de identificador, matriz de permisos por rol).
5. Garantizar la seguridad de la información de los usuarios (CSRF, bcrypt, consultas preparadas, escape de salida, control de errores, cookies httponly y SameSite).
6. Asegurar que la solución sea robusta y libre de errores aplicando pruebas exhaustivas (28 suites end-to-end, 20 casos de verificación, auditoría de sintaxis y de warnings).
7. Optimizar el sitio web para una carga rápida y una experiencia de usuario mejorada (caché de recursos, consultas indexadas, imágenes optimizadas, paginación de listados).

2.4 Alcance y límites del proyecto

Dentro del alcance	Fuera del alcance (y por qué)
Registro e inicio de sesión de clientes; gestión de vehículos (CRUD); agenda de turnos con validación de disponibilidad; historial de servicios; pagos con saldo prepago y por transferencia; factura electrónica; presupuestos; avisos por correo; administración de usuarios, sedes, servicios, marcas y permisos; matriz de permisos editable; datos bancarios; panel de agenda del gestor; despliegue en Docker con respaldos automatizados.	Integración real con pasarelas de pago o con el sistema bancario (el enunciado no lo pide: el pago se reporta y lo valida el taller); envío de correo con SMTP real en producción (el cliente SMTP propio queda configurable por variables de entorno); aplicación móvil nativa; facturación electrónica ante la DGI (se emite un comprobante descargable en PDF, sin valor fiscal).

2.5 Marco regulatorio e institucional

El proyecto se realiza en el marco del **Proyecto de Pasaje de Grado** de la Dirección General de Educación Técnico Profesional (ANEP–UTU), Escuela Técnica de Pando, en el curso 3.º de Tecnologías de la Información. El reglamento del proyecto establece, entre otros, los siguientes puntos, que el equipo cumplió de forma específica:

Exigencia normativa	Cumplimiento en el proyecto
Entrega de letría, conformación de grupos, primera, segunda y tercera entrega con fechas y formatos definidos	Se respeta el calendario oficial; esta documentación corresponde a la segunda y tercera entrega (14 de setiembre de 2026 y presentación del software en la defensa)
Documentación bajo normas APA con carátula	Este documento: carátula institucional, portada, contenido, estructura de capítulos y referencias
Modelado con UML: clases, actividades, secuencia, navegabilidad, despliegue y componentes	Capítulo 5, con los seis diagramas renderizados desde el código real
Documentación de base de datos con DDL, datos de prueba y SQL	Capítulo 6 y archivos `db/schema.sql`, `db/seed.sql`, `db/migracion.sql`
Control de versiones con repositorio (por ejemplo GitHub)	Repositorio propio con 18 commits descriptivos (capítulo 10)
Plan de pruebas: caja negra y caja blanca	Capítulo 9, con la suite completa y sus resultados
Servidor web y de respaldo, script operador de centro de cómputos, entrega de máquina virtual .OVA	Capítulo 11: scripts `instalar.sh`, `respaldar.sh`, `restaurar.sh`, `caida.sh`, `estado.sh` y `operador.sh`
Acta de integración firmada, coordinación de proyecto y actas de reunión	Anexo C: checklist de cumplimiento y pendientes normative

3. Marco teórico y metodología

3.1 Metodología de desarrollo: Scrum

El desarrollo se organizó con un enfoque ágil **Scrum**, con iteraciones cortas (sprints) y entregables verificables al final de cada una. La decisión se tomó porque el enunciado define el producto en términos de "funcionalidades" verificables por rol, lo que se traduce naturalmente a historias de usuario y a criterios de aceptación observables.

Elemento de Scrum	Aplicación concreta en TGS
Product Backlog	Los 40 requisitos del enunciado, agrupados por rol y por módulo, con las cinco brechas detectadas en la revisión intermedia (G1–G5) incorporadas como historias adicionales
Sprint	Ciclos de dos a tres semanas: análisis, modelado, implementación, pruebas y documentación
Roles	Coordinación de proyecto (product owner), desarrollo backend, desarrollo frontend, base de datos e infraestructura, calidad y pruebas
Artefactos	Documento de Requisitos, diagramas UML, MER y diccionario de datos, glosario bilingüe, plan de pruebas, backlog de pendientes, actas de reunión y control de versiones
Definition of Done	Una funcionalidad se considera terminada cuando está implementada, con validación de servidor, con token CSRF, con los textos en los dos idiomas, con prueba automática en verde y con la captura de pantalla en la documentación

3.2 Modelo de arquitectura en tres capas

La aplicación se organizó con una arquitectura de **tres capas**, que separa las responsabilidades y permite que el mismo código sirva a los tres roles sin duplicación:

Capa	Responsabilidad	Archivos
Presentación (front-end)	HTML, CSS y JavaScript de cada pantalla; formularios; validación de cliente; mensajes	42 archivos `.php` de vista, `css/tgs-panel.css`, `css/tema-tgs.css`, `lang/es.php`, `lang/en.php`
Lógica de negocio (back-end)	Autenticación, autorización, sesión, CSRF, reglas de disponibilidad, cálculo de importes, envío de correo, factura	`auth.php`, `correo.php`, `taller_disponibilidad.php`, `funciones_pago.php`, `conexion.php`, `config.php`
Datos	Esquema, consultas preparadas, migraciones, datos de prueba	MySQL 8.4, `db/schema.sql`, `db/seed.sql`, `db/migracion.sql`

El control de acceso se resuelve en la capa de negocio mediante una **matriz de permisos** (29 permisos × 3 roles) declarada en el código y almacenada en la tabla `permiso_rol`, que el administrador puede modificar desde la pantalla `permisos.php`. Cada página pide el permiso que necesita con `verificarPermiso()`; si el usuario no lo tiene, se lo envía a la página de acceso denegado. De esta forma, cambiar un permiso se resuelve en un solo lugar y no exige tocar el código de las pantallas.

3.3 Ingeniería de software y UML

Para modelar el sistema se utilizaron los diagramas UML que el enunciado exige, cada uno con un propósito concreto:

Diagrama	Para qué sirve en este proyecto
Clases	Mostrar los módulos de dominio (Usuario, Vehículo, Taller, Servicio, Reserva, Pago, Presupuesto, Disponibilidad, Correo, Permisos) con sus datos y operaciones
Actividades	Describir el flujo de reserva de un turno, incluida la decisión de disponibilidad
Secuencia	Detallar el flujo de validación de un pago reportado por el cliente
Navegabilidad	Mapear el camino del usuario dentro del sitio según el rol
Despliegue	Mostrar cómo se reparte la aplicación entre contenedores y máquina virtual
Componentes	Separar la interfaz, la lógica de negocio, el acceso a datos y los scripts de operación

Todos los diagramas de este documento se generaron a partir del código y del esquema real de la base de datos, en notación Mermaid (texto que se puede versionar y revisar en el repositorio), y se renderizaron automáticamente al imprimir este documento.

3.4 Fundamentos de seguridad web

El proyecto aplica los controles de seguridad web habituales de OWASP, y cada uno tiene su implementación concreta:

Control	Implementación en el proyecto
Autenticación	Contraseñas con `password_hash()` (bcrypt, `PASSWORD_DEFAULT`) y verificación con `password_verify()`; el error del login es genérico para no revelar si el correo existe
CSRF	Token por sesión generado con `random_bytes(32)`, verificado con `hash_equals()` en **todos** los formularios POST
Inyección SQL	Consultas preparadas con PDO en el 100 % de las operaciones; los identificadores se castean con `intval()` o `FILTER_VALIDATE_INT`
XSS	Salida escapada con `htmlspecialchars()`; el único HTML sin escapar es el `iframe` del mapa, que se guarda y se muestra con control
Control de acceso	Matriz de permisos por rol, comprobada en servidor en cada página y en cada acción sensible
Gestion de cookies	Cookie de sesión con `httponly` y `SameSite=Lax`, `secure` si hay HTTPS, y `session_regenerate_id(true)` tras autenticarse
Filtrado de errores	`display_errors=Off` y `log_errors=On`: el visitante ve un mensaje genérico y el detalle real queda en el log del contenedor
Gestión de secretos	Credenciales fuera del código: variables de entorno (`.env` versionado con valores de ejemplo y `.env.local` local, nunca versionado)

3.5 Metodología de pruebas

Se combinaron las dos familias de prueba que pide el enunciado:

- **Caja negra (funcionales):** se verifican las salidas del sistema a partir de las entradas del usuario, sin depender del código. La herramienta es `curl` sobre la aplicación real en Docker, con un script reutilizable que inicia sesión real y obtiene el token CSRF de cada página. Se escribieron **28 suites** que cubren los 40 requisitos y **20 casos** adicionales de verificación de las brechas G1–G5.
- **Caja blanca (estructurales):** se revisa el código y la base: auditoría de sintaxis de los 42 archivos PHP, búsqueda de consultas concatenadas, verificación de que todo formulario POST lleva token CSRF, revisión de que no queden páginas con errores PHP y control de que la base no tenga filas huérfanas ni claves foráneas rotas.

4. Requisitos del sistema

Los requisitos se extrajeron literalmente del *Documento Técnico: Requerimientos de Software para Plataforma de Servicios Mecánicos* del enunciado y se contrastaron con la implementación, señalando en la tabla el estado real de cada uno y el lugar del código donde queda resuelto.

4.1 Requisitos funcionales del cliente

#	Requisito del enunciado	Estado	Resolución en el código
1	Registro con correo, clave, nombre completo y tipo de vehículo, nombre completo y tipo de vehículo; devolución de la información registrada	**Cumplido**	`registro.php`: alta con validación, hash bcrypt y devolución de los datos; el tipo de vehículo se solicita al registrar el vehículo (`reservas.php`)
2	Inicio de sesión con correo y clave; acceso al panel principal	**Cumplido**	`login.php` con `password_verify()` y redirección según rol: `panel_cliente.php`, `dashboard_gestor.php`, `dashboard_admin.php`
3.1	Agregar vehículo	**Cumplido**	`reservas.php`, acción `agregar_vehiculo`
3.2	Consultar vehículos	**Cumplido**	`panel_cliente.php`, sección «Mis vehículos»
3.3	Editar vehículo	**Cumplido**	`reservas.php`, acción `editar_vehiculo`
3.4	Eliminar vehículo	**Cumplido**	`reservas.php`, acción `eliminar_vehiculo` con confirmación
3.5	Historial de servicios del cliente	**Cumplido**	`historial_cliente.php`: por vehículo, con sede, fecha, detalle, costo, cobros y enlace a la factura
4.1	Visualización de talleres en el mapa	**Cumplido**	`index.php` (portada) y `taller.php` (detalle de sede con `iframe` de Google Maps)
4.2	Filtrar talleres por marca, tipo de servicio, etc.	**Cumplido**	Selectores de marca y de tipo de servicio en la portada y en el panel, sobre las tablas puente `taller_marca` y `taller_servicio`
4.3	Solicitud de servicio mecánico	**Cumplido**	`reservas.php`: turno con vehículo, servicio, sede, fecha y hora
4.4	Seleccionar taller y reservar horario disponible	**Cumplido**	Selector de sede + horarios libres calculados en vivo por `disponibilidad.php` y validados de nuevo en el servidor al confirmar
5.1	Lista de talleres con ubicación y estado (lugares disponibles o no)	**Cumplido**	Tarjetas de sede con dirección, mapa, horario semanal, capacidad y **lugares libres hoy**
5.2	Vista detallada de un taller con opciones para reservar o cancelar	**Cumplido**	`taller.php?id=N`: datos, mapa, marcas, servicios, horario, disponibilidad y los turnos propios para cancelar
6.1	Historial: taller, fechas, detalle del servicio y costo	**Cumplido**	`historial_cliente.php` con el costo de cada visita y sus cobros
7.1	Pagar el servicio directamente desde la aplicación	**Cumplido**	Pago con **saldo prepago** (queda pagado al instante) o **reporte de pago** con comprobante, que valida el taller

7.2	Métodos de pago disponibles (tarjeta, cuenta prepaga, transferencia)	**Cumplido**	`efectivo`, `tarjeta`, `transferencia` y `prepago` (columna `usuario.saldo_prepago`)
7.3	Factura electrónica descargable al concretar un pago	**Cumplido**	`factura.php`: se emite al aprobar el pago, numerada `F-AAAA-NNNN`, imprimible a PDF
8.1	Modificar datos personales, clave y contacto	**Cumplido**	`perfil.php`, con validación de la clave actual para cambiarla
8.2	Dar de baja la cuenta	**Cumplido**	`eliminar_cuenta.php`: pide la clave y confirmación; borra el usuario y sus datos por cascada
8.3	Consultar los servicios agendados	**Cumplido**	`panel_cliente.php` y `reservas.php`

4.2 Requisitos funcionales del gestor de puntos de carga

Requisito del enunciado	Estado	Resolución en el código
Gestión de servicios (CRUD): crear, leer, actualizar y eliminar talleres con ubicación, dirección, medios de pago, RUT, cantidad de vehículos, tipos de vehículo y servicios ofrecidos	**Cumplido**	`administrar_talleres.php` (listado y filtros), `agregar_taller.php` (alta con horario semanal, marcas y servicios), `editar_taller.php` (edición) y borrado con `?eliminar=ID`
Visualización: mapa integrado con la ubicación de todos los talleres	**Cumplido**	Cada tarjeta de sede incluye su mapa; la portada muestra el listado completo con filtros
Filtros por ubicación, tipo de servicio y estado	**Cumplido**	Filtro por texto (nombre, dirección, correo o teléfono), por marca y por tipo de servicio, con `LIKE ... ESCAPE` para tratar los comodines como texto
Usuario de taller: listar los vehículos a los que se le está realizando servicio	**Cumplido**	`agenda_gestor.php` y `detalle_reserva.php`: turno del día con vehículo, cliente y servicio
Subir el presupuesto a la plataforma y enviar un correo avisando que está disponible	**Cumplido**	`administrar_presupuestos.php` (alta con ítems, precios y total) + `correo.php` (aviso al cliente)
Registrar los servicios realizados	**Cumplido**	Estado de la reserva («Finalizada») con auditoría en `reserva_historial` y notificación asociada
Notificar al cliente la finalización del servicio por mail y por plataforma	**Cumplido**	`notificaciones.php`: cola de avisos por correo con estado, número de intentos y respuesta del transporte

4.3 Requisitos funcionales del administrador

Requisito del enunciado	Estado	Resolución en el código
Gestión de usuarios (CRUD) con búsqueda y filtros	**Cumplido**	`administrar_usuarios.php`: alta, edición, baja, filtro por rol y por texto
Gestión de tipos de usuario (roles) con permisos configurables	**Cumplido**	`permisos.php`: matriz de 29 permisos × 3 roles editable en caliente, con los permisos críticos del administrador protegidos
Listado de los tipos de usuario y de la cantidad de usuarios con reservas	**Cumplido**	Resumen de la matriz con el conteo de usuarios por rol y sus reservas
Gestión de talleres de servicio (CRUD)	**Cumplido**	Idem cliente y gestor: el administrador tiene los mismos módulos y permisos adicionales
Dashboard del administrador	**Cumplido**	`dashboard_admin.php`: diez accesos a los módulos de gestión

4.4 Requisitos no funcionales

Requisito	Meta	Cómo se cumple
Seguridad de la información	Sin vulnerabilidades conocidas en el modelo de amenazas del proyecto	Consultas preparadas, CSRF, bcrypt, escape de salida, control de acceso por permiso, cookies protegidas y gestión de errores sin filtrar datos internos
Usabilidad	Interfaz intuitiva para un usuario no técnico	Un panel por rol, mensajes de error en lenguaje claro, confirmaciones antes de acciones destructivas y ayuda contextual en cada formulario
Disponibilidad	El sitio debe responder dentro del horario de atención	Contenedores con *healthcheck* de MySQL, reinicio automático de Docker y política de respaldos con restauración probada
Portabilidad	Instalación en cualquier equipo con Docker	`scripts/instalar.sh`: deja el sistema listo con un comando, a partir de `docker-compose.yml` y las variables de entorno
Rendimiento	Carga rápida de los listados	Índices en todas las claves foráneas y de búsqueda, consultas que evitan N+1, paginación de listados largos y recursos gráficos comprimidos
Mantenibilidad	Código legible y modifications controladas	Comentarios en cada archivo, una única conexión a la base, matriz de permisos centralizada y control de versiones con commits descriptivos
Internacionalización	Interfaz en español e inglés	Archivos de textos por idioma y selector ES/EN en las 24 pantallas, con persistencia en la sesión y glosario bilingüe (capítulo 12)
Accesibilidad	Usable con teclado ylector de pantalla	Etiquetas asociadas a cada control, `aria-label` en los selectores, contraste alto sobre fondo oscuro y mensajes de estado con texto y no solo color

4.5 Matriz de trazabilidad

La siguiente tabla vincula cada bloque de requisitos con la pantalla, la base de datos y la prueba automatizada que lo verifican. Es el instrumento que permite, durante la defensa, responder «cómo saben que está hecho».

Requisito	Pantalla(s)	Tabla(s) de la base	Prueba
1 – 2. Registro y login	`registro.php`, `login.php`	`usuario`	`test_alta.sh`, `test_perfil.sh`
3 – 3.4. Vehículos (CRUD)	`reservas.php`, `panel_cliente.php`	`vehiculo`	`test_editar.sh`, `test_ownership.sh`
3.5 / 6.1. Historial de servicios	`historial_cliente.php`	`reserva`, `pago`, `factura`	`test_gaps.sh` (G2)
4.1 / 5.1. Mapa y ubicación	`index.php`, `taller.php`	`taller`	`test_nav.sh`
4.2 / 4.4. Filtros por marca y servicio	`index.php`, `administrar_talleres.php`	`taller_marca`, `taller_servicio`	`test_filtros.sh`, `test_filtros2.sh` (G4)
4.3 / 4.4. Reserva de turno	`reservas.php`, `procesar_reserva.php`	`reserva`, `reserva_historial`	`test_disp.sh`, `test_repro.sh`
5.2. Vista detallada de sede	`taller.php`	`taller`, `taller_horario`	`test_gaps.sh` (G3)
7.1 / 7.2. Pagos y métodos	`pagos_cliente.php`, `administrar_pagos.php`	`pago`, `usuario.saldo_prepago`	`test_pagos.sh`, `test_pagos2.sh`, `test_pago_cliente.sh`, `test_a_pagar.sh` (G5)
7.3. Factura electrónica	`factura.php`	`factura`	`test_gaps.sh` (G1)
8.1 / 8.2. Perfil y baja	`perfil.php`, `eliminar_cuenta.php`	`usuario`	`test_perfil.sh`, `test_baja.sh`
9.1 / 9.2. Presupuestos	`presupuestos.php`, `administrar_presupuestos.php`	`presupuesto`, `presupuesto_detalle`	`test_presupuestos.sh`
Gestor: agenda y servicios	`agenda_gestor.php`, `detalle_reserva.php`	`reserva`, `reserva_historial`	`test_agenda.sh`, `test_detalle.sh`, `test_modal.sh`
Gestor: avisos al cliente	`notificaciones.php`	`notificacion`	`test_notificaciones.sh`, `test_smtp.sh`
Admin: usuarios, roles y permisos	`administrar_usuarios.php`, `permisos.php`	`usuario`, `permiso_rol`	`test_permisos.sh`, `test_matriz.sh`, `test_filtros3.sh`
Admin: datos bancarios	`datos_pago.php`	`datos_pago`	`test_datos_pago.sh`
Seguridad transversal	Todas	—	`test_csrf.sh`, `test_ownership.sh`, auditoría de warnings

4.6 Reglas de negocio y tablas de decisión

El enunciado pide construir «árboles o tablas de decisión que contemplen las reglas de negocio, como la asignación de talleres según disponibilidad y estado». Las reglas implementadas se expresan con las siguientes tablas de decisión, que se implementan en el servidor (el JavaScript solo informa; nunca autoriza).

Tabla de decisión 1: disponibilidad de una franja horaria

Condición	Resultado
La sede no tiene horario configurado para ese día de la semana	Franja no disponible
La franja queda fuera del horario de atención (`hora_inicio` – `hora_fin`)	Franja no disponible
La cantidad de reservas **vivas** de esa franja (excluidas las canceladas) es menor que la **capacidad** de la sede	Franja disponible
La cantidad de reservas vivas es igual o mayor que la capacidad	Franja no disponible

Tabla de decisión 2: asignación del turno a una sede

Estado de la reserva previa	Condición	Decisión
Nueva	La sede atiende la marca del vehículo y ofrece el servicio pedido	Se asigna a esa sede
Programada	El cliente solicita reprogramar a otra fecha/hora	Se revalida la disponibilidad de la nueva franja; si no hay lugar, se rechaza y no se modifica la reserva
Programada	El cliente cancela	La reserva pasa a «Cancelada» y libera el lugar de la franja
En curso	El gestor la marca como finalizada	Pasa a «Finalizada», se registra el costo en `reserva.precio_total` y se emite el aviso al cliente
Cualquiera	Se solicita una sede o servicio que la sede no ofrece	Se rechaza con mensaje explicativo

Tabla de decisión 3: estado del cobro

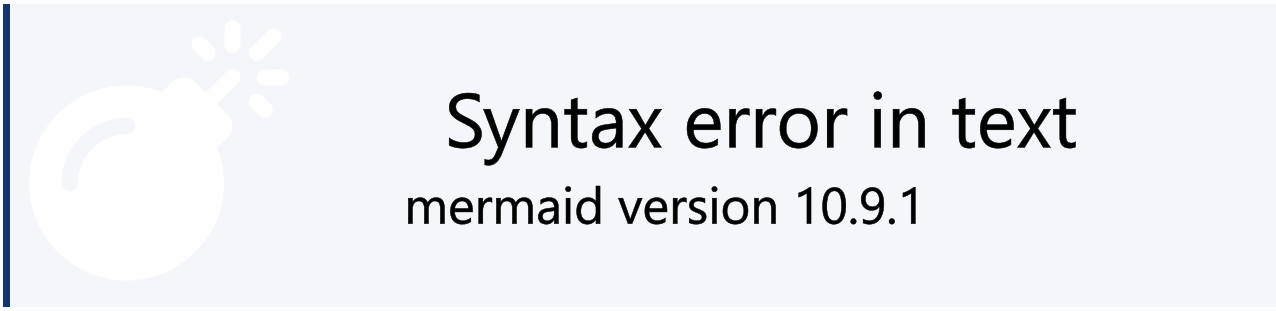
Hecho	Nuevo estado de la reserva
Se crea la reserva	`pendiente`
El cliente paga con su saldo prepago	`pagado` (de inmediato) y se descuenta el saldo
El cliente reporta un pago con comprobante	Sigue `pendiente` o pasa a `parcial`: el cobro queda `reportado` a la espera de validación
El gestor aprueba el cobro reportado	Se recalcula: `pagado` si la suma de cobros cubre el total, `parcial` si no
El gestor rechaza o anula un cobro	Se recalcula el estado a partir de los cobros válidos y se libera el saldo

5. Diseño del sistema

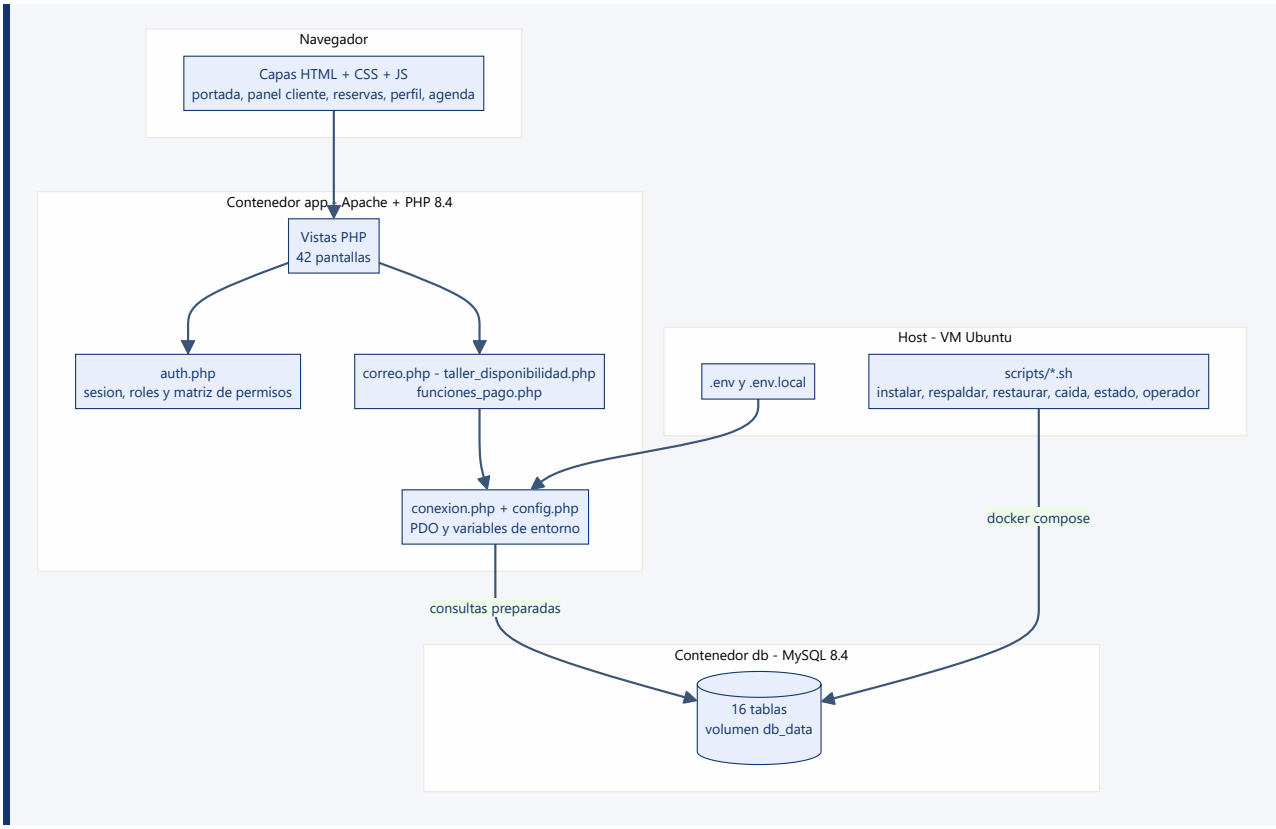
Este capítulo contiene los seis diagramas UML exigidos por el enunciado. Todos se generaron a partir del código y del esquema real y se renderizan automáticamente desde la notación Mermaid versionada en el repositorio (`docs/ENTREGABLES-TECNICOS.md`).

5.1 Diagrama de despliegue

Muestra cómo se reparte la aplicación entre el navegador, los dos contenedores y la máquina virtual, y dónde persisten los datos.

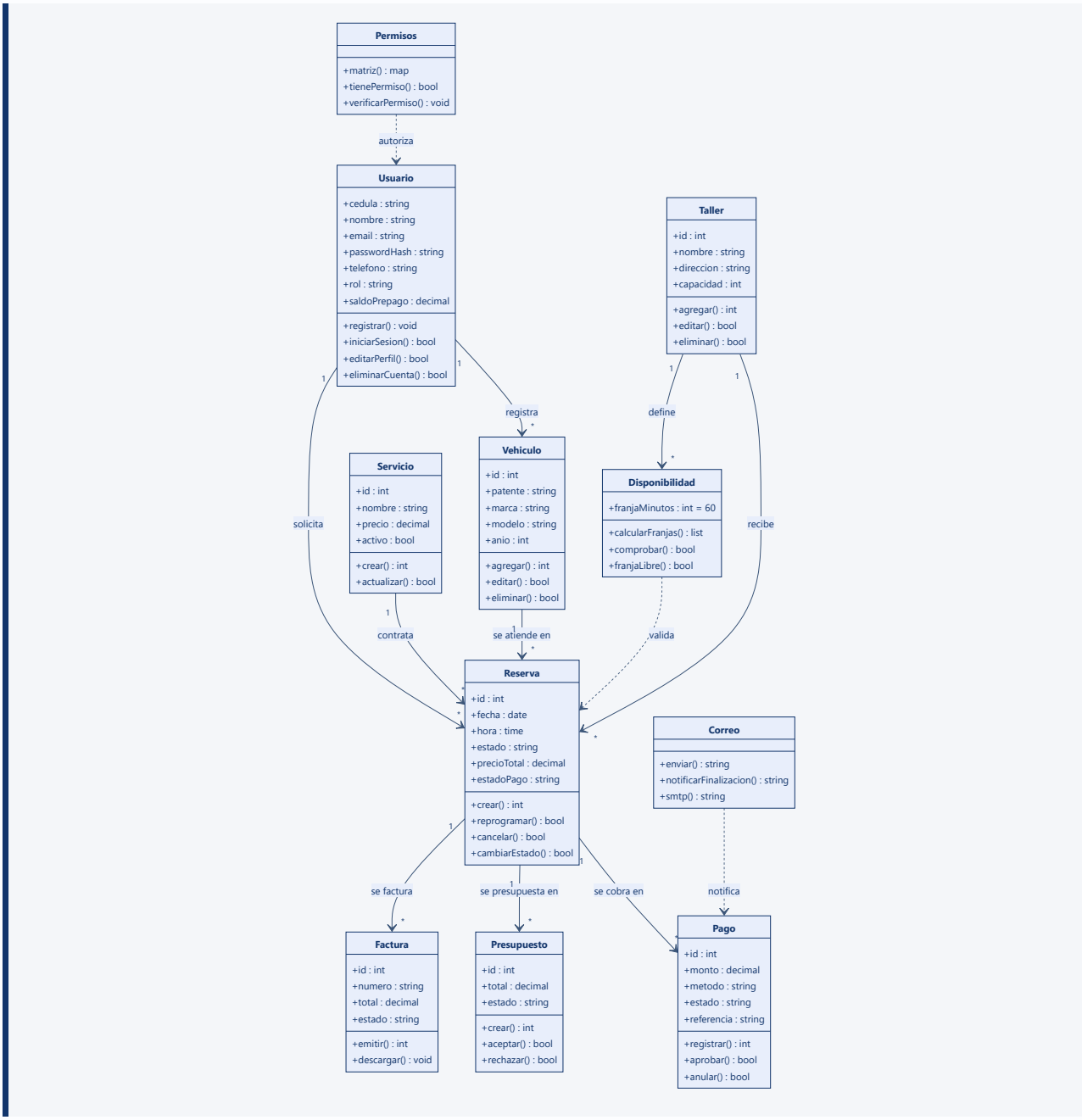


5.2 Diagrama de componentes



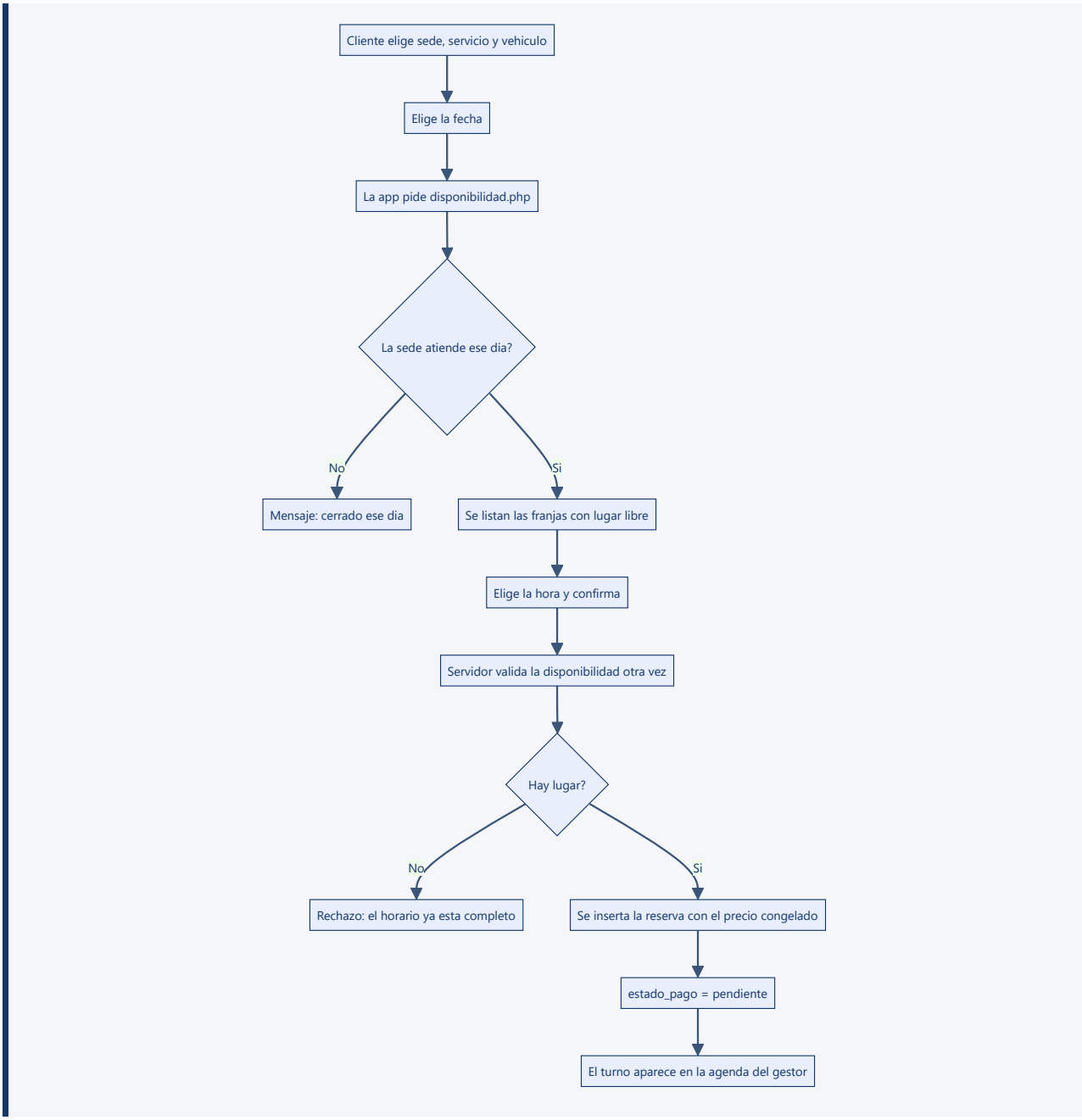
5.3 Diagrama de clases

El proyecto es procedural con funciones de dominio (sin framework ni clases), por lo que cada bloque del diagrama representa un módulo del código con sus datos y sus operaciones: es la equivalencia funcional que pide el enunciado.



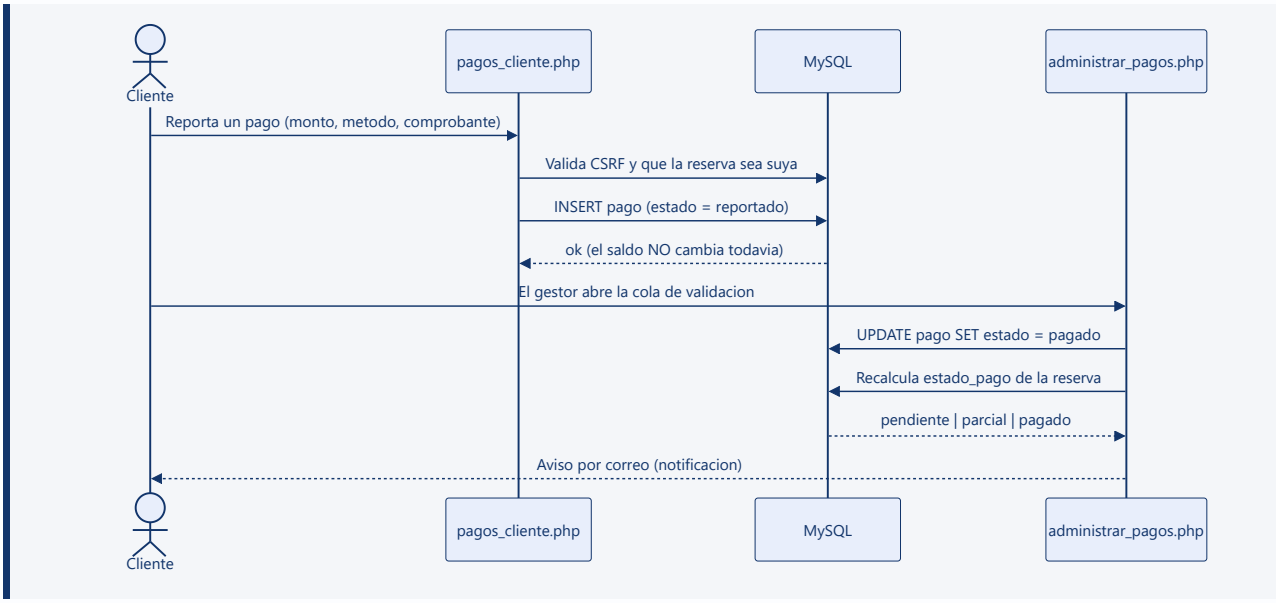
5.4 Diagrama de actividades

Flujo completo de reserva de un turno, con la decisión de disponibilidad resuelta en el servidor.

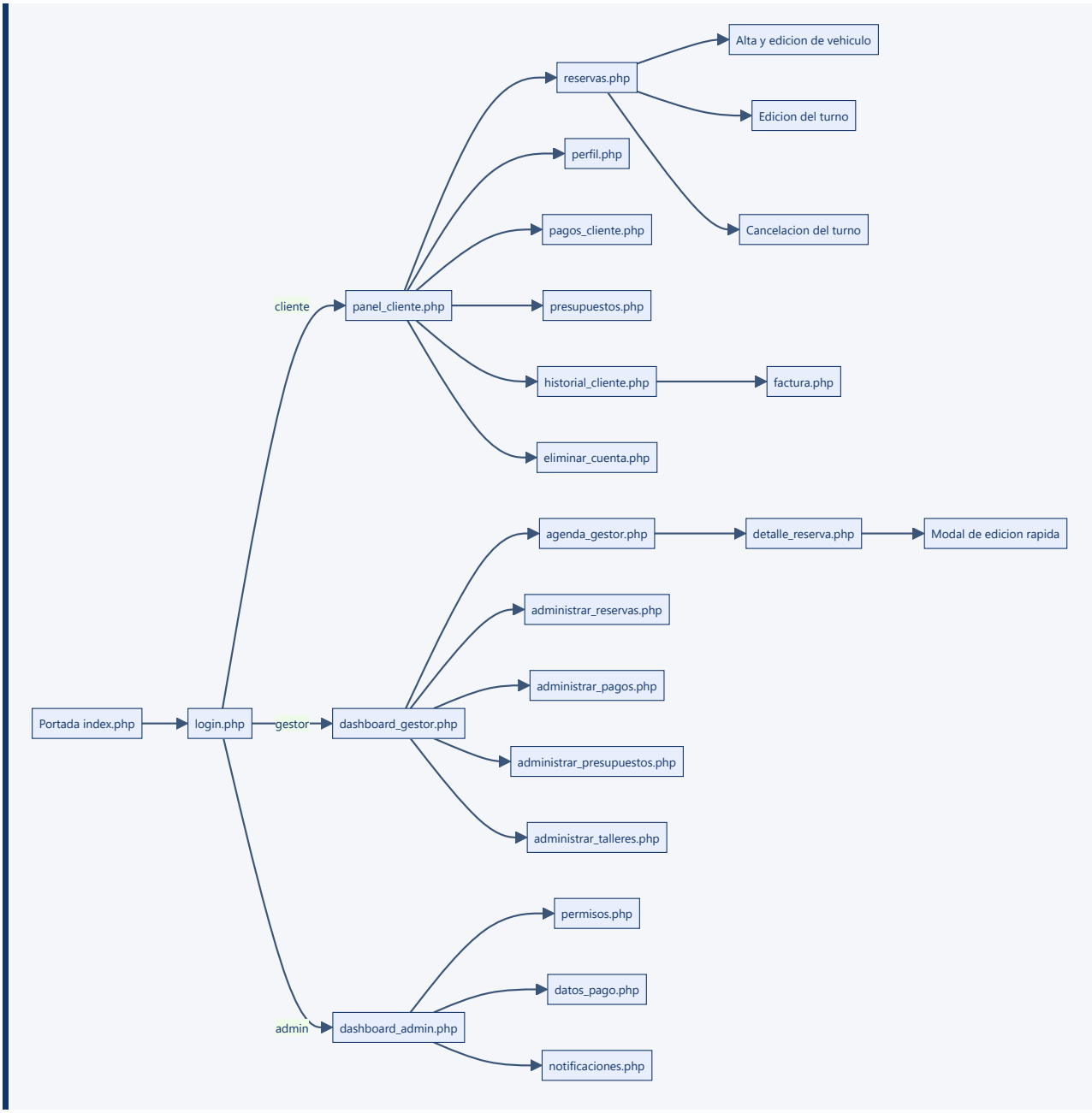


5.5 Diagrama de secuencia

Validación de un pago reportado por el cliente: hasta que el taller lo aprueba, el estado de cobro de la reserva no cambia.


Cliente
pagos_cliente.php
MySQL
administrar_pagos.php
Reporta un pago (monto, metodo, comprobante)
Valida CSRF y que la reserva sea suya
INSERT pago (estado = reportado)
ok (el saldo NO cambia todavia)
El gestor abre la cola de validacion
UPDATE pago SET estado = pagado
Recalcula estado_pago de la reserva
pendiente | parcial | pagado
Aviso por correo (notificacion)
Cliente
pagos_cliente.php
MySQL
administrar_pagos.php

5.6 Diagrama de navegabilidad



5.7 Criterios de diseño de la interfaz de usuario

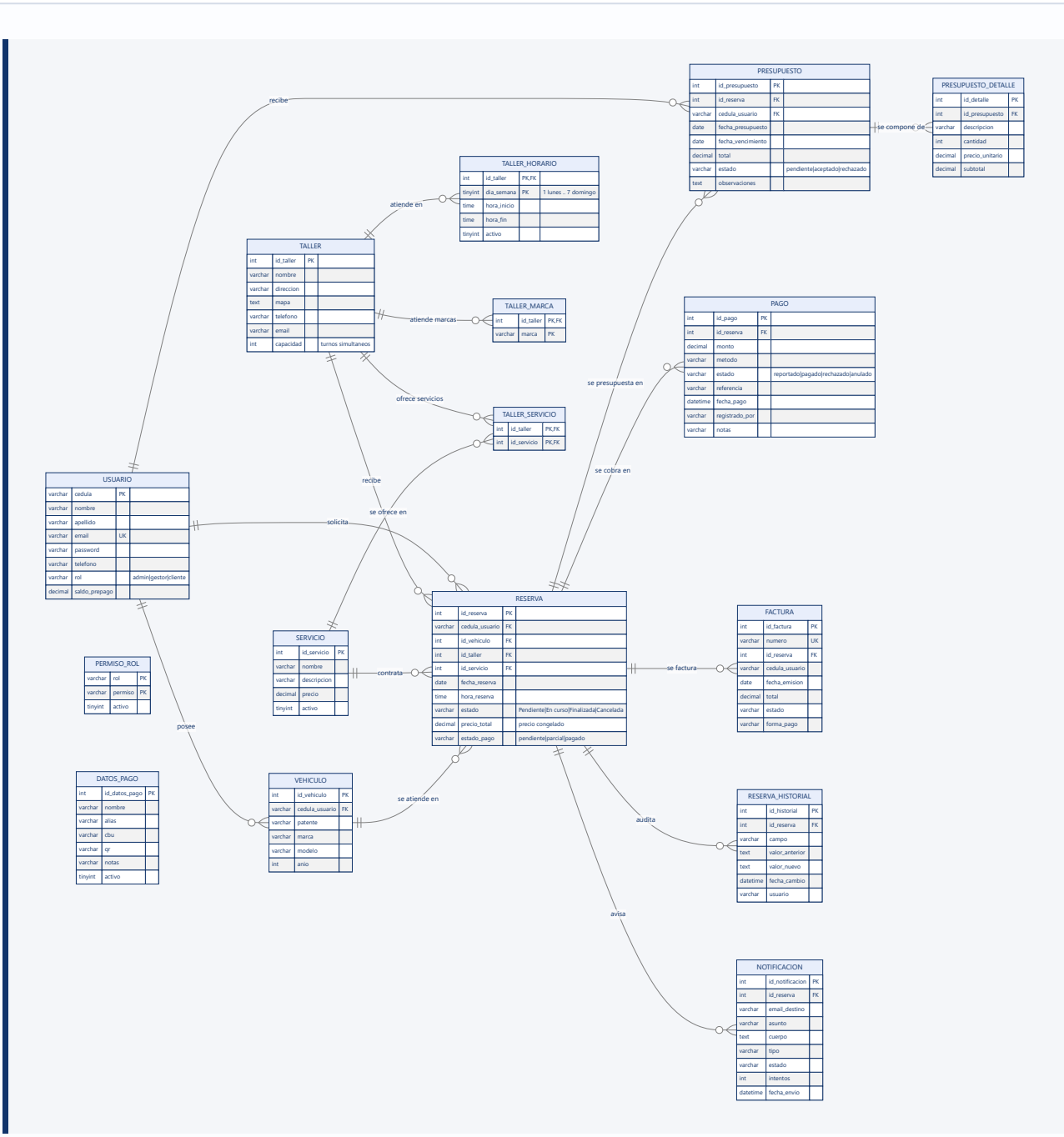
Los criterios de diseño de interfaz que el enunciado pide explicitar se resumen así:

Criterio	Decisión de diseño y aplicación
Consistencia	Una sola hoja de estilo compartida (`css/tgs-panel.css`) define la paleta, los botones, las tarjetas y las tablas; las 42 pantallas usan la misma estructura de cabecera, contenido y pie
Jerarquía visual	Cada pantalla tiene un título de acción, un subtítulo que explica la función y un bloque principal; las acciones destructivas van separadas de las principales
Realimentación	Toda acción del usuario produce una respuesta visible: mensaje de éxito, de error o de confirmación antes de borrar
Carga cognitiva	Un panel por rol con las tarjetas agrupadas por módulo; las acciones del cliente están en un solo lugar (panel, turnos, pagos, presupuestos, historial)
Minimización de errores	Validación en el cliente y en el servidor, campos con valores por defecto razonables, confirmaciones y mensajes que indican cómo corregir el problema
Reconocimiento sobre recuerdo	Los valores frecuentes (sedes, servicios, métodos de pago) se ofrecen en listas desplegables con el valor ya visible, en lugar de recordar códigos
Accesibilidad	Contraste alto, foco visible, textos de al menos 14 píxeles en el cuerpo, `aria-label` en controles sin etiqueta visible y alternativa textual a los mapas
Idioma	Selector ES/EN en todas las pantallas, con la traducción centralizada y el idioma conservado durante la sesión

6. Base de datos

La base de datos se implementó en **MySQL 8.4** con motor InnoDB, juego de caracteres `utf8mb4_unicode_ci` y claves foráneas con acción `ON DELETE CASCADE`, de modo que al eliminar un usuario o una sede se borren en cascada sus datos dependientes y no queden filas huérfanas. El esquema definitivo está en `db/schema.sql`, los datos de prueba en `db/seed.sql` y la actualización del esquema en `db/migracion.sql`.

6.1 Modelo Entidad-Relación



Cardinalidades. Un usuario tiene 0..N vehículos, 0..N reservas y 0..N presupuestos; cada reserva pertenece a exactamente un usuario, un vehículo, una sede y un servicio. Una reserva tiene 0..N cobros

(puede ser 0 si todavía no pagó, o 2 si pagó un anticipo y luego el saldo) y 0..1 factura. Un presupuesto tiene 1..N ítems. Una sede tiene 0..N horarios (uno por día de la semana), 0..N marcas y 0..N servicios.

6.2 Diccionario de datos

La base tiene **16 tablas**. `PK` = clave primaria, `FK` = clave foránea, `UK` = único.

6.2.1 `usuario`

Columna	Tipo	Nulo	Clave	Default	Descripción
cedula	varchar(20)	no	**PK**		Cédula de identidad; identifica al usuario en toda la base
nombre	varchar(100)	no			Nombre
apellido	varchar(100)	sí			Apellido
email	varchar(150)	no	**UK**		Correo de acceso; identificador del login
password	varchar(255)	no			Hash bcrypt (nunca el texto plano)
telefono	varchar(30)	sí			Teléfono de contacto
rol	varchar(50)	no		'cliente'	`admin`, `gestor` o `cliente`
saldo_prepago	decimal(10,2)	no		0.00	Saldo de la cuenta prepaga del cliente

Índices: `PRIMARY (cedula)`, `UNIQUE (email)`, `usuario_rol_idx (rol)`.

6.2.2 `vehiculo`

Columna	Tipo	Nulo	Clave	Default	Descripción
id_vehiculo	int	no	**PK**	AI	Identificador del vehículo
cedula_usuario	varchar(20)	no	**FK**		→ `usuario.cedula` (CASCADE)
patente	varchar(20)	no			Dominio de matrícula
marca	varchar(50)	no			Toyota, Volkswagen...
modelo	varchar(50)	no			Corolla, Gol...
anio	int	sí			Año del modelo

6.2.3 `taller`, `taller_horario`, `taller_marca` y `taller_servicio`

Tabla	Columna	Tipo	Clave	Descripción
taller	id_taller	int	**PK** AI	Identificador de la sede
	nombre	varchar(100)		Nombre comercial
	direccion	varchar(200)		Dirección
	mapa	text		Código HTML (iframe) del mapa
	telefono	varchar(30)		Teléfono
	email	varchar(100)		Correo
	capacidad	int		Turnos simultáneos que acepta (2)
taller_horario	id_taller	int	**PK**,**FK**	→ `taller.id_taller` (CASCADE)
	dia_semana	tinyint	**PK**	1 = lunes … 7 = domingo
	hora_inicio	time		Apertura de la sede ese día
	hora_fin	time		Cierre de la sede ese día
taller_marca	id_taller	int	**PK**,**FK**	Sede que atiende la marca
	marca	varchar(50)	**PK**	Marca de vehículo atendida
taller_servicio	id_taller	int	**PK**,**FK**	Sede que ofrece el servicio
	id_servicio	int	**PK**,**FK**	Servicio que ofrece

6.2.4 `servicio` y `reserva`

Tabla	Columna	Tipo	Clave	Descripción
servicio	id_servicio	int	**PK** AI	Identificador del servicio
	nombre	varchar(120)		Nombre del servicio
	precio	decimal(10,2)		Precio de lista (0.00)
	activo	tinyint(1)		Solo los activos se pueden reservar
reserva	id_reserva	int	**PK** AI	Identificador del turno
	cedula_usuario	varchar(20)	**FK**	→ `usuario.cedula`
	id_vehiculo	int	**FK**	→ `vehiculo.id_vehiculo`
	id_taller	int	**FK**	→ `taller.id_taller`
	id_servicio	int	**FK**	→ `servicio.id_servicio`
	fecha_reserva	date		Día del turno
	hora_reserva	time		Hora del turno
	estado	varchar(50)		Pendiente / En curso / Finalizada / Cancelada
	precio_total	decimal(10,2)		Precio **congelado** al reservar (base del cobro)
	estado_pago	varchar(20)		pendiente / parcial / pagado

6.2.5 `pago`, `factura` y `datos_pago`

Tabla	Columna	Tipo	Clave	Descripción
pago	id_pago	int	**PK** AI	Identificador del cobro
	id_reserva	int	**FK**	→ `reserva.id_reserva` (CASCADE)
	monto	decimal(10,2)		Importe del cobro (0.00)
	metodo	varchar(30)		efectivo / tarjeta / transferencia / prepago
	estado	varchar(20)		reportado / pagado / rechazado / anulado
	referencia	varchar(60)		N.º de comprobante o transferencia
	fecha_pago	datetime		Momento del cobro
	registrado_por	varchar(20)		Cédula de quien lo registró o validó
	notas	varchar(255)		Observación
factura	id_factura	int	**PK** AI	Identificador
	numero	varchar(30)	**UK**	Número correlativo F-AAAA-NNNN
	id_reserva	int	**FK**	→ `reserva.id_reserva`
	cedula_usuario	varchar(20)		Cliente facturado
	fecha_emision	date		Fecha de emisión
	total	decimal(10,2)		Importe facturado (0.00)
	estado	varchar(20)		emitida / anulada
	forma_pago	varchar(30)		Método con que se pagó
datos_pago	id_datos_pago	int	**PK** AI	Identificador
	nombre	varchar(100)		Nombre de la cuenta
	alias	varchar(60)		Alias bancario
	cbu	varchar(34)		CBU
	qr	varchar(500)		URL o <img> del código QR
	notas	varchar(255)		Indicaciones para el cliente
	activo	tinyint(1)		Si está visible para el cliente

6.2.6 `presupuesto`, `presupuesto_detalle`, `reserva_historial`, `notificacion` y `permiso_rol`

Tabla	Columna	Tipo	Clave	Descripción
presupuesto	id_presupuesto	int	**PK** AI	Identificador
	id_reserva	int	**FK**	→ `reserva.id_reserva` (CASCADE)
	cedula_usuario	varchar(20)	**FK**	→ `usuario.cedula` (CASCADE)
	fecha_presupuesto	date		Fecha de emisión
	fecha_vencimiento	date		Vencimiento de la propuesta
	total	decimal(10,2)		Suma de los ítems (caché de consulta)
	estado	varchar(20)		pendiente / aceptado / rechazado
	observaciones	varchar(255)		Notas para el cliente
	fecha_respuesta	datetime		Cuándo respondió el cliente
	respondido_por	varchar(20)		Cédula de quien respondió
	creado_por	varchar(20)		Cédula de quien lo generó
presupuesto_detalle	id_detalle	int	**PK** AI	Identificador del ítem
	id_presupuesto	int	**FK**	→ `presupuesto.id_presupuesto` (CASCADE)
	descripcion	varchar(255)		Concepto (repuesto, mano de obra)
	cantidad / precio_unitario / subtotal	int / decimal / decimal		Cantidad, precio por unidad y subtotal
reserva_historial	id_historial	int	**PK** AI	Identificador
	id_reserva	int	**FK**	→ `reserva.id_reserva`
	campo	varchar(50)		Campo modificado (fecha, hora, estado...)
	valor_anterior	text		Valor previo
	valor_nuevo	text		Valor nuevo
	fecha_cambio / usuario	datetime / varchar(100)		Momento del cambio y quién lo hizo
notificacion	id_notificacion	int	**PK** AI	Identificador
	id_reserva	int	**FK**	→ `reserva.id_reserva` (CASCADE)
	cedula_usuario	varchar(20)		Destinatario interno
	email_destino	varchar(150)		Correo al que se envió
	asunto / cuerpo	varchar(200) / text		Asunto y mensaje
	tipo	varchar(40)		finalizacion / reprogramacion / general
	estado	varchar(20)		pendiente / enviado / error
	respuesta_transporte	varchar(250)		Lo que respondió el servidor de correo
	intentos	int		Cuántos intentos se hicieron

	fecha_envio / created_at	datetime / timestamp		Cuándo se envió
permiso_rol	rol	varchar(50)	**PK**	admin / gestor / cliente
	permiso	varchar(80)	**PK**	Nombre del permiso (29 definidos)
	activo	tinyint(1)		0 = ese rol no tiene el permiso
	updated_at	timestamp		Última modificación

6.3 Normalización

1FN — valores atómicos. No hay grupos repetidos: los ítems de un presupuesto están en `presupuesto_detalle` (no en una columna `items`) y los horarios de una sede en `taller_horario` (no en columnas `lunes, martes, …`). Cada campo guarda un solo valor: `reserva.fecha_reserva` y `reserva.hora_reserva` están separados, igual que `vehiculo.marca` y `vehiculo.modelo`.

2FN — dependencia de toda la clave. Las tablas con clave compuesta son `taller_horario` (`id_taller`, `dia_semana`), `permiso_rol` (`rol`, `permiso`), `taller_marca` y `taller_servicio`. En todas ellas cada atributo depende de la **clave completa**: en `taller_horario`, `hora_inicio` y `hora_fin` dependen del día **y** de la sede, y no existe ningún atributo que dependa solo de `id_taller` (eso vive en `taller`). Las tablas de clave simple ya están en 2FN por definición.

3FN — sin dependencias transitivas. `reserva.precio_total` es un dato **congelado** a propósito: si dependiera de `servicio.precio` (que puede cambiar), el historial de lo cobrado se rompería al modificar la lista de precios. Congelarlo rompe la dependencia transitiva `reserva → servicio → precio` y da trazabilidad histórica. Del mismo modo, `pago` no guarda ni el nombre del cliente ni el de la sede: se llega por `pago → reserva → usuario/taller`, sin duplicar datos.

BCNF y 4FN. Todas las dependencias funcionales son de clave completa, por lo que el esquema cumple BCNF; y no existen dependencias multivaluadas fuera de las tablas de detalle, por lo que cumple 4FN.

6.4 Consultas, vistas y roles de la base de datos

Los reportes que pide el enunciado («historial de servicios de un vehículo o la facturación por taller») se resuelven con `JOIN` entre varias tablas y, en los casos que lo requieren, con subconsultas que DEVUELVEN valores agregados. Un ejemplo real del historial del cliente:

```
SELECT r.id_reserva, r.fecha_reserva, r.hora_reserva, r.estado,
       r.precio_total, t.nombre AS taller, s.nombre AS servicio,
       v.marca, v.modelo, v.patente
  FROM reserva r
  JOIN taller   t ON t.id_taller   = r.id_taller
  JOIN servicio s ON s.id_servicio = r.id_servicio
  JOIN vehiculo v ON v.id_vehiculo = r.id_vehiculo
 WHERE r.cedula_usuario = ?
 ORDER BY r.fecha_reserva DESC, r.hora_reserva DESC;
```

Y la facturación por sede, que acumula los cobros válidos de cada taller:

```
SELECT t.nombre AS taller,
       COUNT(DISTINCT r.id_reserva)              AS turnos,
       SUM(r.precio_total)                       AS facturado,
       SUM(COALESCE(p.total_cobrado, 0))         AS cobrado,
       SUM(r.precio_total - COALESCE(p.total_cobrado, 0)) AS saldo
  FROM taller t
  JOIN reserva r ON r.id_taller = t.id_taller
  LEFT JOIN (SELECT id_reserva, SUM(monto) AS total_cobrado
               FROM pago WHERE estado = 'pagado' GROUP BY id_reserva) p
    ON p.id_reserva = r.id_reserva
 GROUP BY t.id_taller, t.nombre
 ORDER BY facturado DESC;
```

La aplicación también ofrece **subconsultas** en las pantallas de agenda y de cobros para mostrar el total adeudado sin recalcularlo en PHP, y la base incluye **vistas SQL** que dejan los cuatro reportes complejos disponibles para cualquier consulta externa:

Vista	Qué reporta
`v_historial_servicios`	Historial por cliente y vehículo: sede, servicio, estado, precio, cobrado y saldo pendiente
`v_facturacion_por_taller`	Por sede: turnos, clientes, facturado, cobrado y saldo pendiente
`v_ocupacion_sedes`	Reservas por sede, día y franja, comparadas con la capacidad (lugares libres)
`v_cobros_por_metodo`	Cobros agrupados por método de pago y estado, con el primero y el último

Las vistas están definidas en `db/vistas.sql` y se aplican también desde `db/migracion.sql`, de modo que una base ya instalada no se queda sin sus reportes. Se crean con `CREATE OR REPLACE VIEW`, por lo que el script es idempotente. Un ejemplo de uso:

```
SELECT cliente_nombre, patente, taller_nombre, servicio_nombre,
       precio_total, total_cobrado, saldo_pendiente
  FROM v_historial_servicios
 WHERE cedula = '11111111'
 ORDER BY fecha_reserva DESC;
```

6.4.1 Roles de usuario en el gestor de base de datos

El enunciado pide «creación de roles de usuario en el gestor de base de datos para acceder a MySQL de forma segura». El script `scripts/crear_roles_bd.sh` deja tres perfiles de acceso, cada uno con los permisos justos para su uso:

Rol de MySQL	Permisos	Para qué se usa
`tgs`	Todo sobre la base del proyecto	La aplicación web (lo usa `config.php`)
`tgs_lectura`	Solo `SELECT`	Consultas de informe y auditoría: puede ver los datos pero no puede escribirlos
`tgs_admin`	Todo sobre la base y lectura de `mysql.*`	Administración de la base desde el script operador

El propio script verifica la propiedad de seguridad: intenta escribir con el rol de lectura y comprueba que MySQL se lo niegue, de modo que la demostración queda automatizada y no depende de que alguien la haga a mano.

6.5 DDL, datos de prueba y migración

El esquema se entrega como **DDL** en `db/schema.sql` (16 tablas con sus claves, índices y restricciones), y los datos de prueba como un conjunto de `INSERT` en `db/seed.sql` (5 usuarios de los tres roles con sus vehículos, 2 sedes con horario, marcas y servicios, 3 servicios, reservas en distintos estados, pagos, presupuestos y una cuenta bancaria). El proceso de instalación carga ambos archivos de forma automática la primera vez que se crea el volumen de datos.

Además, `db/migracion.sql` es un script **idempotente** que aplica cambios de esquema sin perder datos (agrega columnas, crea tablas nuevas y agrega índices solo si no existen). Esto permitió evolucionar el modelo durante el proyecto sin perder la información ya cargada, que es justamente lo que exige el enunciado cuando pide registrar la documentación de los cambios.

> **Ejemplo de uso.** El siguiente fragmento muestra cómo se añade una columna sin duplicarla en una segunda ejecución:
>
> ```
> ALTER TABLE usuario
>   ADD COLUMN IF NOT EXISTS saldo_prepago DECIMAL(10,2) NOT NULL DEFAULT 0.00;
> ```

7. Implementación del sistema

7.1 Stack tecnológico y estructura del proyecto

Capa	Tecnología	Versión / detalle
Lenguaje de servidor	PHP	8.4 sobre Apache 2.4 (imagen oficial `php:8.4-apache`), sin framework: `require_once` y funciones de dominio
Base de datos	MySQL	8.4 con InnoDB, `utf8mb4_unicode_ci`
Acceso a datos	PDO	Consultas preparadas, `ERRMODE_EXCEPTION`, una única conexión reutilizada
Contenedores	Docker + Compose	Contenedor `tgs_app` (web) y `tgs_db` (base), red propia, volumen `tgs_db_data` y *healthcheck* de MySQL
Sistema anfitrión	Ubuntu	26.04 LTS en máquina virtual (VirtualBox), IP de la red local, puerto 8080 publicado
Estilos	CSS3	Hoja compartida `tgs-panel.css` y tema institucional `tema-tgs.css`
Idiomas	PHP + archivos de textos	`lang/es.php` y `lang/en.php` con glosario bilingüe
Correo	SMTP	Cliente SMTP propio en `correo.php` (EHLO, AUTH LOGIN/PLAIN, MAIL FROM, RCPT TO, DATA), sin dependencias externas
Control de versiones	Git + GitHub	Repositorio `github.com/ttugol909674-cpu/index`, rama `main`, 18 commits
Pruebas	Bash + curl + MySQL	28 suites end-to-end sobre la aplicación real en Docker

El repositorio se organiza de la siguiente manera:

```
tgs/
|-- .env / .env.example        Variables de entorno (plantilla y valores por defecto)
|-- .env.local                 Configuracion local de la maquina (NUNCA se versiona)
|-- Dockerfile                 Imagen del servidor web (PHP 8.4 + Apache)
|-- docker-compose.yml         Orquestacion de los dos contenedores
|-- config.php                 Lectura del entorno, conexion PDO y ajustes de PHP
|-- conexion.php               Enlace PDO y preparacion idempotente del esquema
|-- auth.php                    Sesion, roles y matriz de permisos (29 permisos)
|-- correo.php                  Cliente SMTP y avisos al cliente
|-- taller_disponibilidad.php   Calculo de franjas libres y validacion de capacidad
|-- funciones_pago.php          Cobros, estado de cobro y emision de facturas
|-- lang/es.php, lang/en.php    Textos de la interfaz en los dos idiomas
|-- css/                        Hojas de estilo (tema TGS y panel)
|-- db/schema.sql               DDL de las 16 tablas
|-- db/seed.sql                 Datos de prueba
|-- db/migracion.sql            Migracion idempotente del esquema
|-- db/vistas.sql               Vistas SQL para los reportes
|-- scripts/                    instalar, respaldar, restaurar, caida, estado,
                                operador (centro de computos) y crear_roles_bd
|-- docs/                       Documentacion tecnica, guia de defensa y diagramas
`-- 42 pantallas .php           Portada, panels y modulos por rol
```

7.2 Backend: PHP, PDO, sesiones y CSRF

Todo el acceso a datos pasa por una única conexión PDO definida en `config.php` y reutilizada por `conexion.php`. El acceso se hace siempre con consultas preparadas, nunca concatenando datos del usuario:

```
$stmt = $conexion->prepare('SELECT * FROM usuario WHERE email = ? AND rol = ?');
$stmt->execute([$email, $rol]);
$usuario = $stmt->fetch(PDO::FETCH_ASSOC);
```

La sesión se inicia con cookie `httponly` y `SameSite=Lax`, y el identificador se regenera al iniciar sesión para evitar la fijación de sesión. El token CSRF se genera una vez por sesión y se valida en cada POST:

```
function csrfToken(): string {
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}

function validarCsrf($tokenRecibido): bool {
    if (empty($_SESSION['csrf_token']) || !is_string($tokenRecibido) || $tokenRecibido === '') {
        return false;
    }
    return hash_equals($_SESSION['csrf_token'], $tokenRecibido);
}
```

La autorización se hace con permisos y no con roles, lo que permite reconfigurar el sistema desde la interfaz sin tocar el código:

```
function verificarPermiso(string $permiso): void {
    exigirSesion();
    if (!tienePermiso($permiso)) {
        header('Location: error_403.php');
        exit();
    }
}
```

7.3 Interfaz, bilingüismo y accesibilidad

La interfaz usa una paleta institucional azul sobre fondo oscuro, común a las 42 pantallas mediante `css/tgs-panel.css`. El bilingüismo se resuelve con archivos de textos por idioma y un selector ES/EN en la cabecera de cada pantalla; el idioma elegido se guarda en la sesión, de modo que se conserva al navegar y sobrevive al inicio de sesión. Una auditoría automática verifica que las 24 pantallas revisadas no mezclen idiomas y que todas tengan el selector.

En accesibilidad se suelen aplicar: contraste alto, foco visible, etiquetas asociadas a los campos, textos alternativos en los mapas y mensajes de estado expresados con texto (no solo con color).

7.4 Paneles por rol

El sistema presenta tres paneles distintos según el rol de la sesión. El panel del administrador se muestra a continuación.

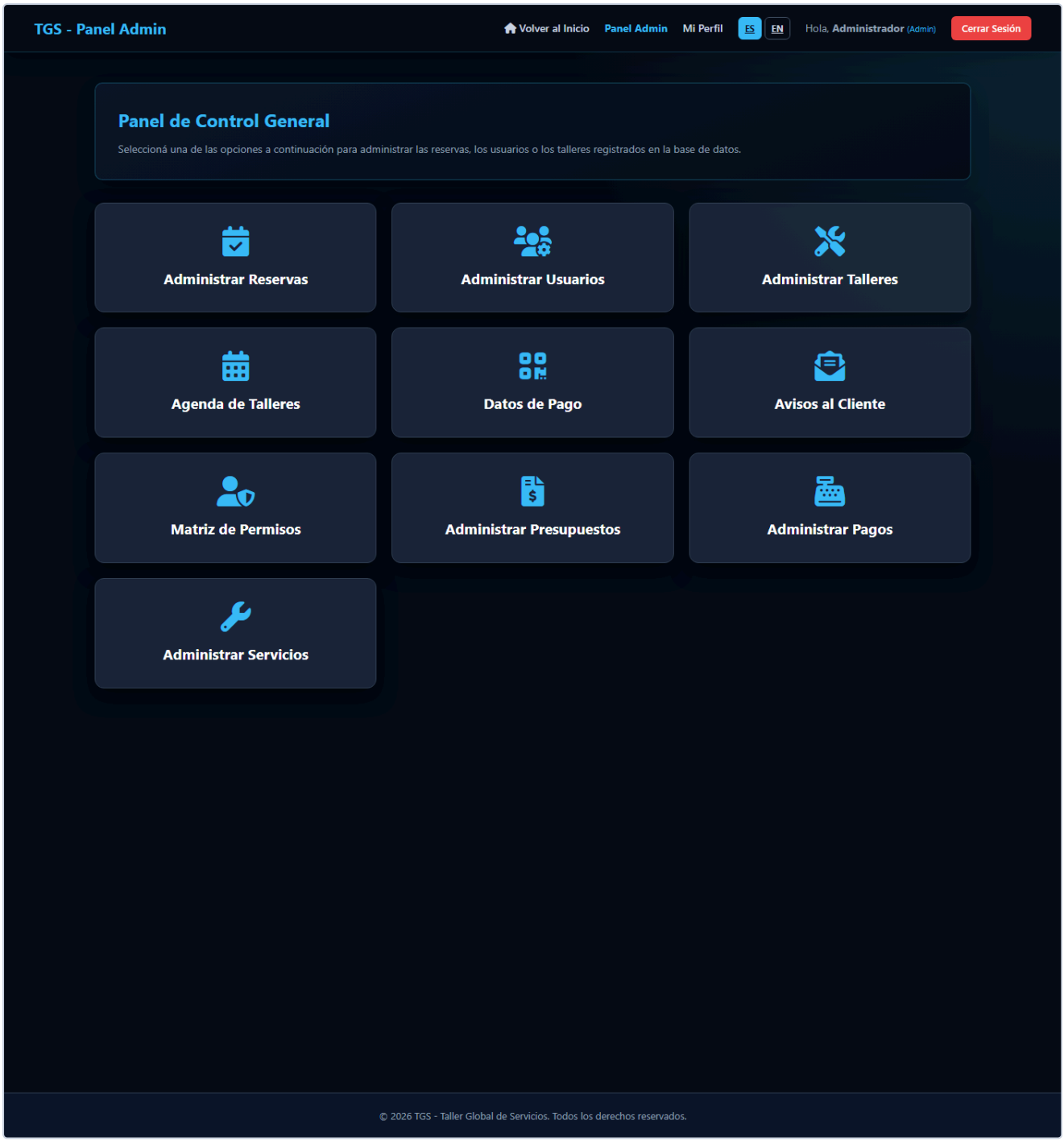


Figura 1. Panel del administrador (`dashboard_admin.php`): diez accesos a los módulos de gestión, con el selector de idioma y la sesión activa.

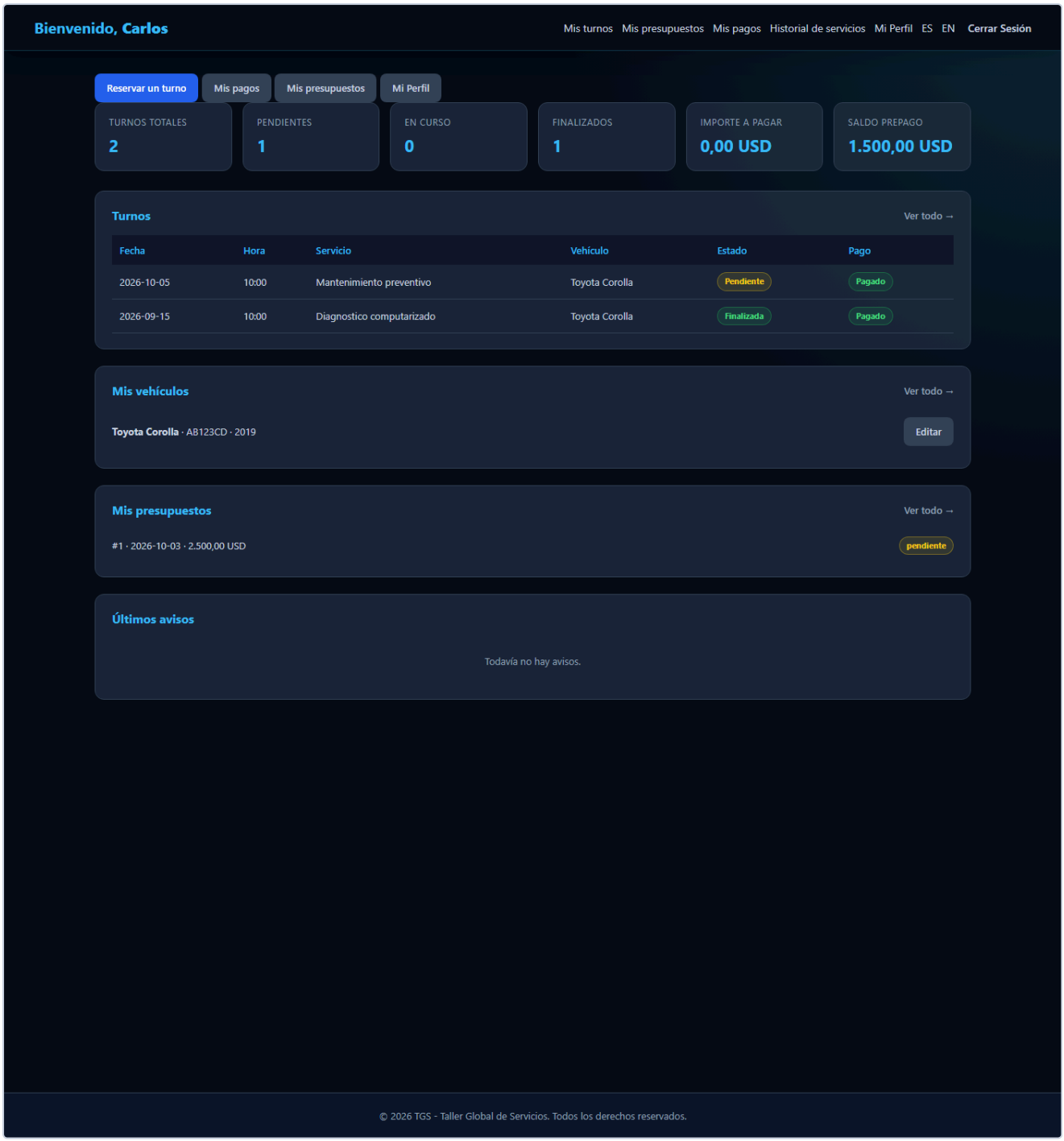


Figura 2. Panel del cliente (`panel_cliente.php`): resumen de la cuenta, turnos, vehículos, presupuestos, pagos, saldo prepago y últimos avisos.



Figura 3. Panel del gestor (`dashboard_gestor.php`): agenda del día, reservas, pagos, presupuestos, sedes y servicios.

7.5 Reservas y disponibilidad

El corazón del sistema es el módulo de disponibilidad (`taller_disponibilidad.php`). Cada sede tiene un horario semanal (`taller_horario`) y una capacidad de turnos simultáneos (`taller.capacidad`). Las franjas son de 60 minutos desde la hora de apertura y la disponibilidad se calcula contando las reservas **vivas** de esa franja: las canceladas no ocupan lugar.

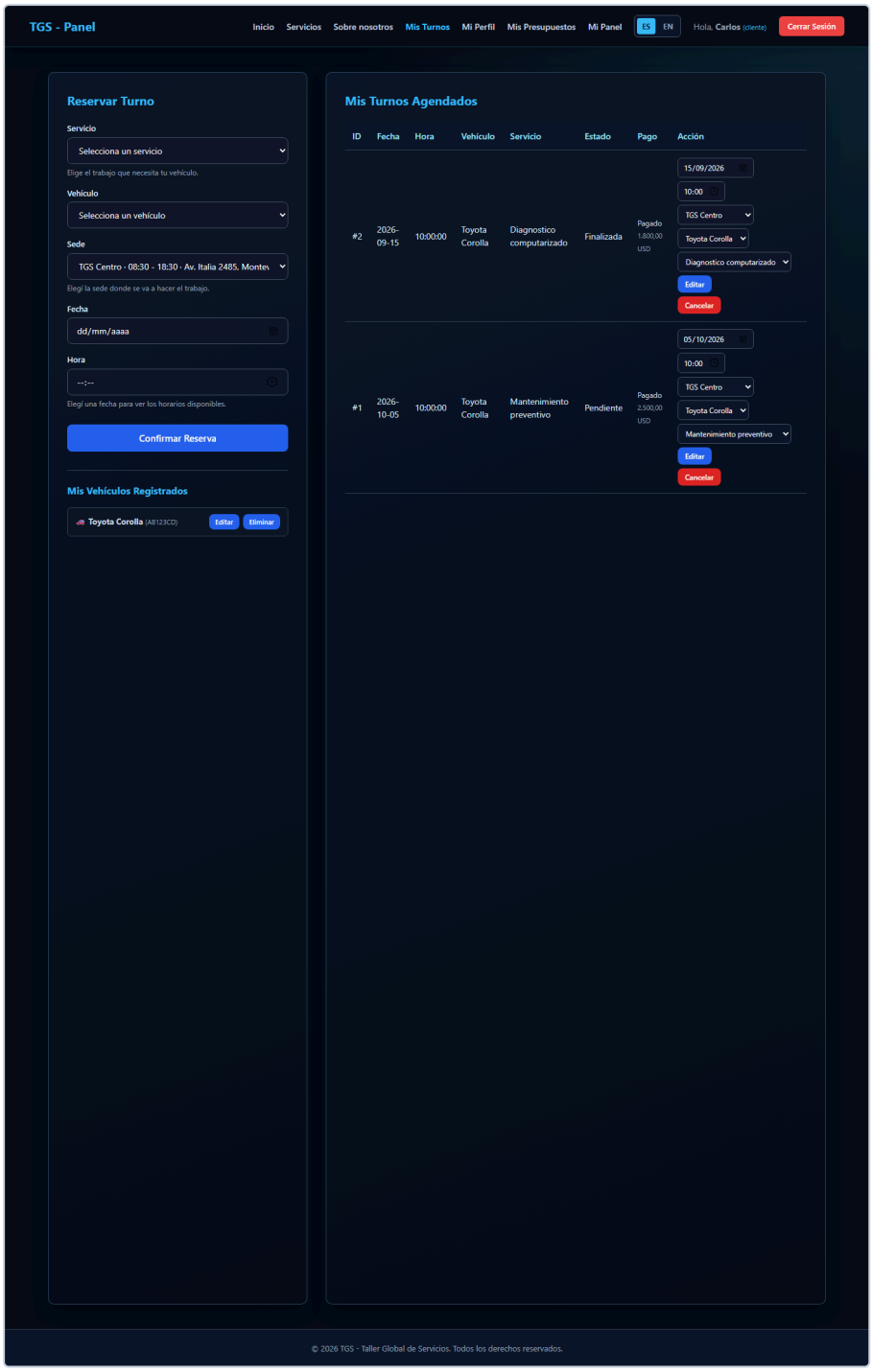


Figura 4. Pantalla de turnos del cliente (`reservas.php`): alta y edición de vehículos, reserva de un turno con selector de sede, servicio, fecha y hora libre.

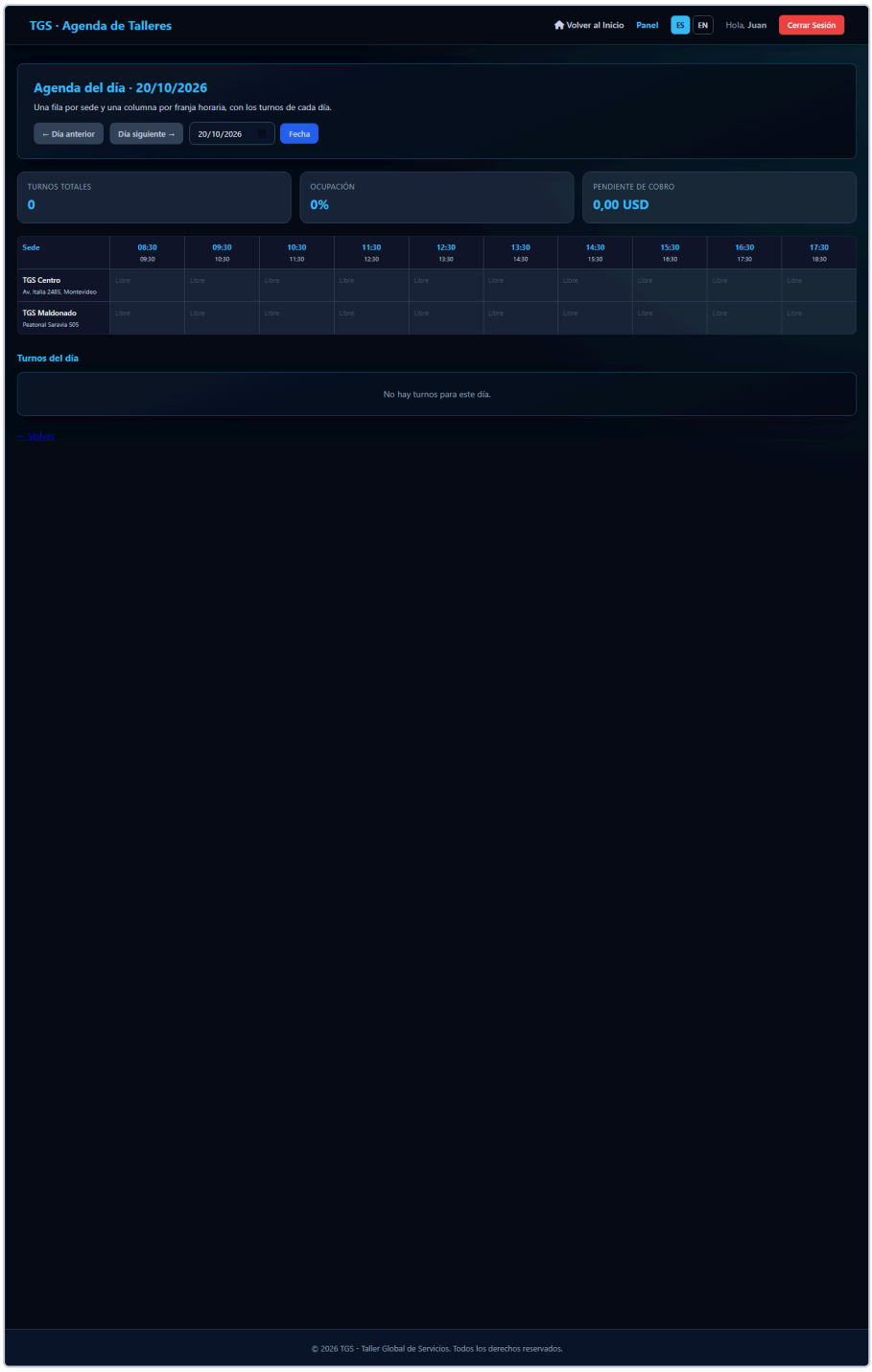


Figura 5. Agenda diaria del gestor (`agenda_gestor.php`): turnos del día con el estado de cada servicio y las franjas libres de cada sede.

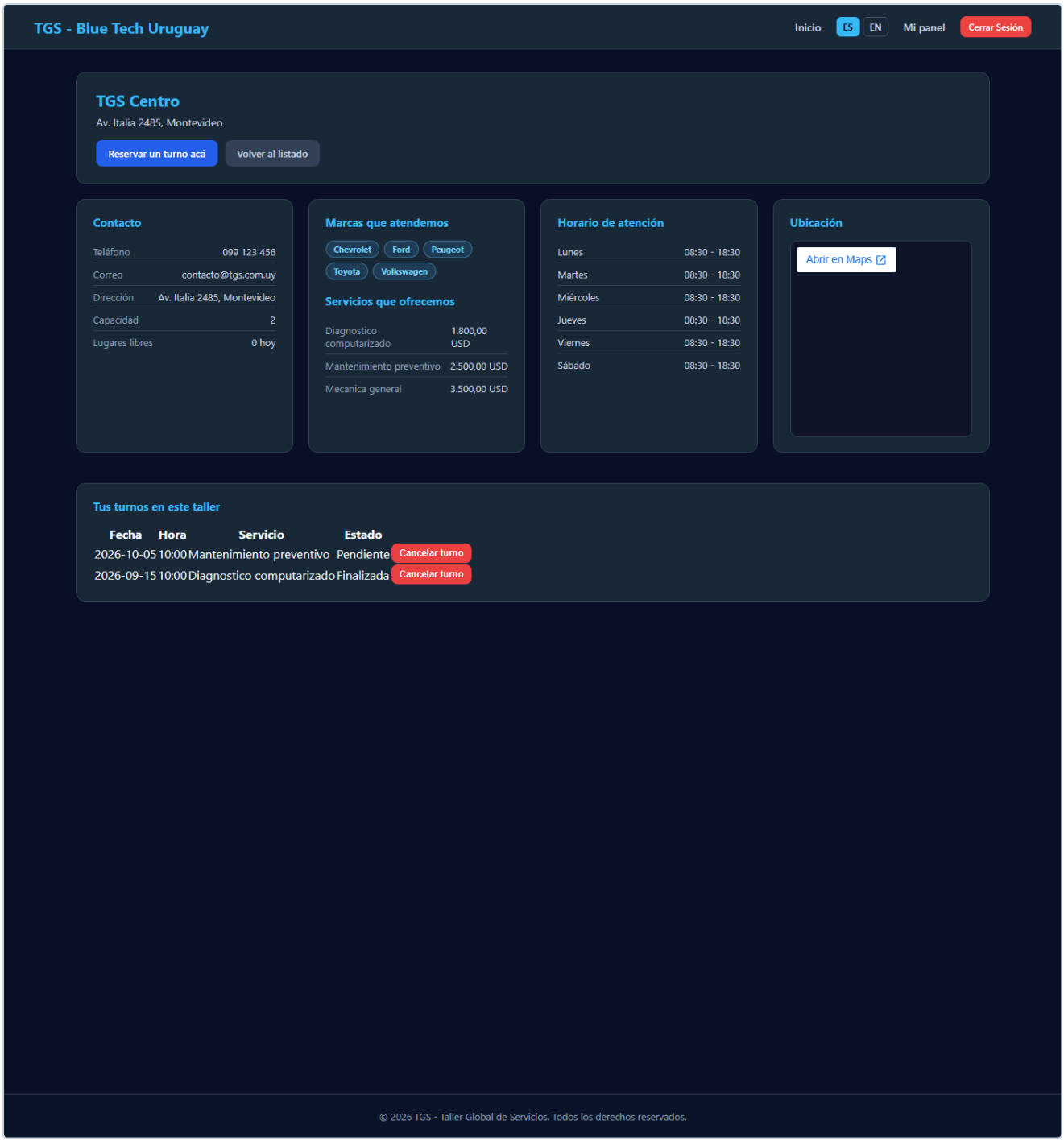


Figura 6. Vista detallada de una sede (`taller.php`): ubicación en el mapa, marcas y servicios atendidos, horario semanal, lugares libres y los turnos propios para cancelar.

7.6 Pagos, presupuestos y facturación

El módulo de pagos implementa el circuito completo: el cliente ve su saldo, los datos bancarios de la empresa con el código QR y el importe pendiente; puede pagar con su **saldo prepago** (queda pagado al instante) o **reportar** un pago con el comprobante; el gestor valida o rechaza ese cobro desde la cola de cobros reportados, y en ese momento se recalcula el estado de la reserva y se emite la factura.

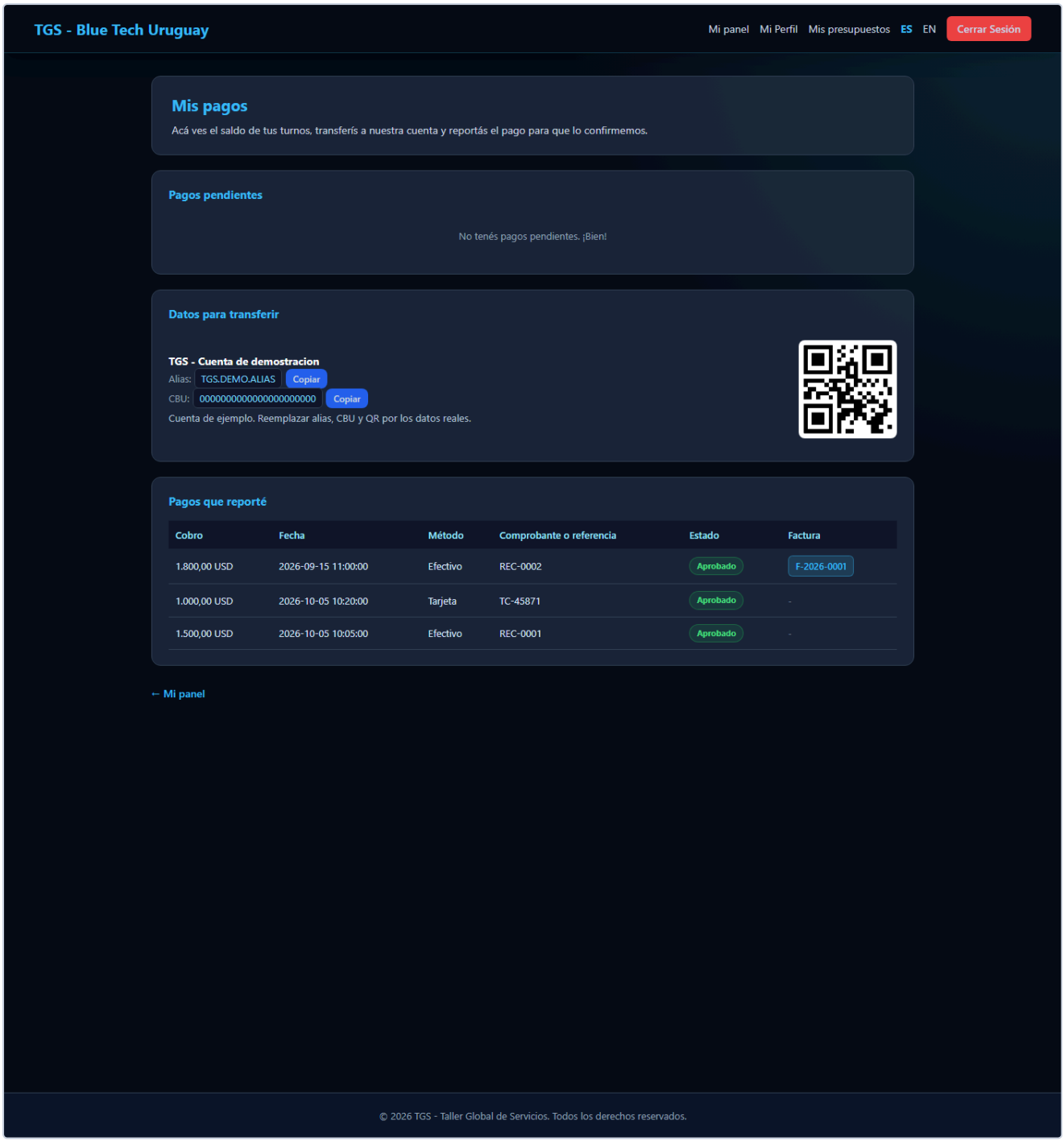


Figura 7. Pantalla de pagos del cliente (`pagos_cliente.php`): saldo prepago, importe pendiente, datos bancarios con QR, reportar un pago y consultar el histórico de cobros.

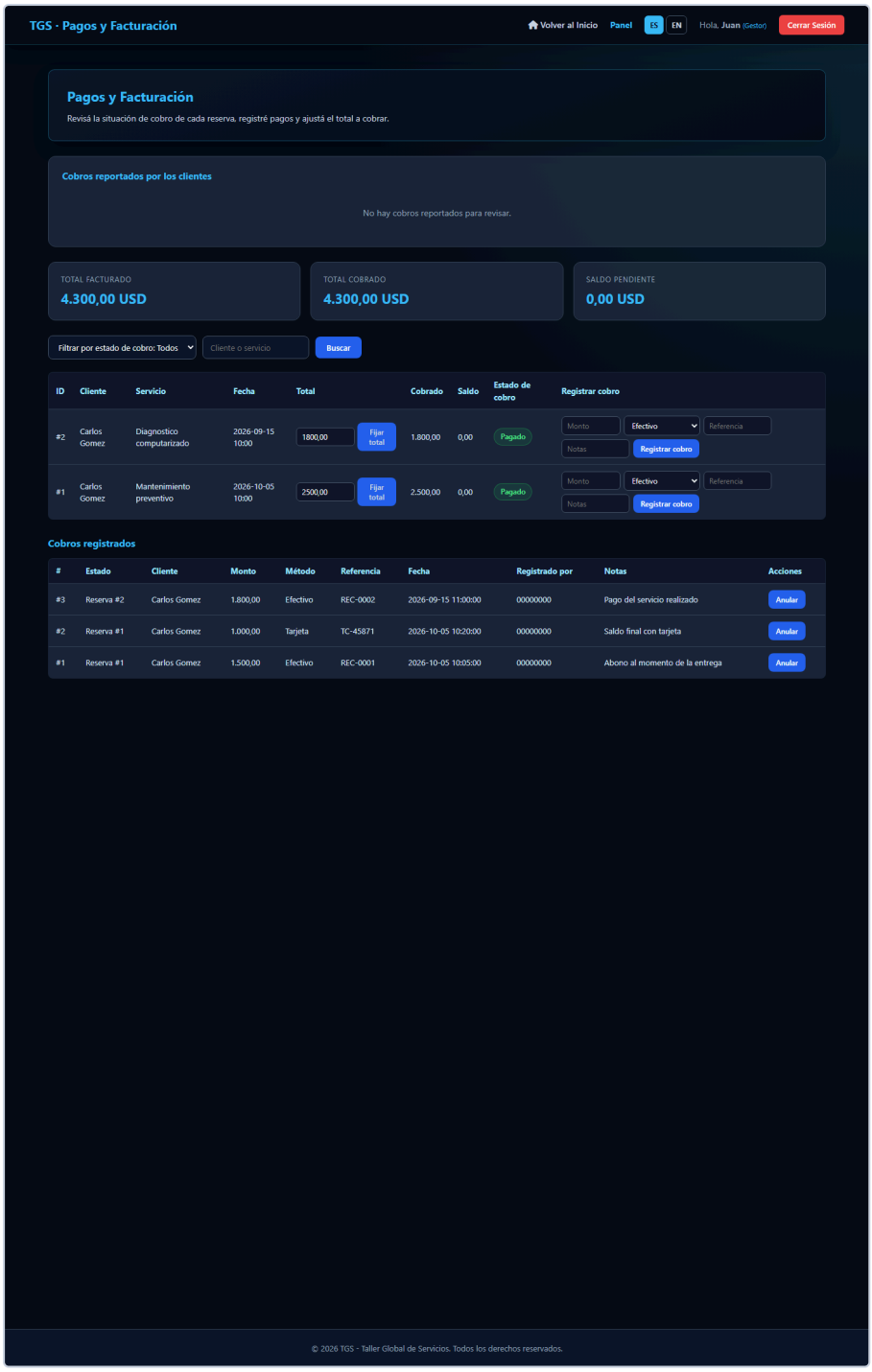


Figura 8. Cola de cobros del gestor (`administrar_pagos.php`): cobros reportados por los clientes para aprobar o rechazar, registro de cobros, ajuste del importe y resumen de facturado y cobrado.

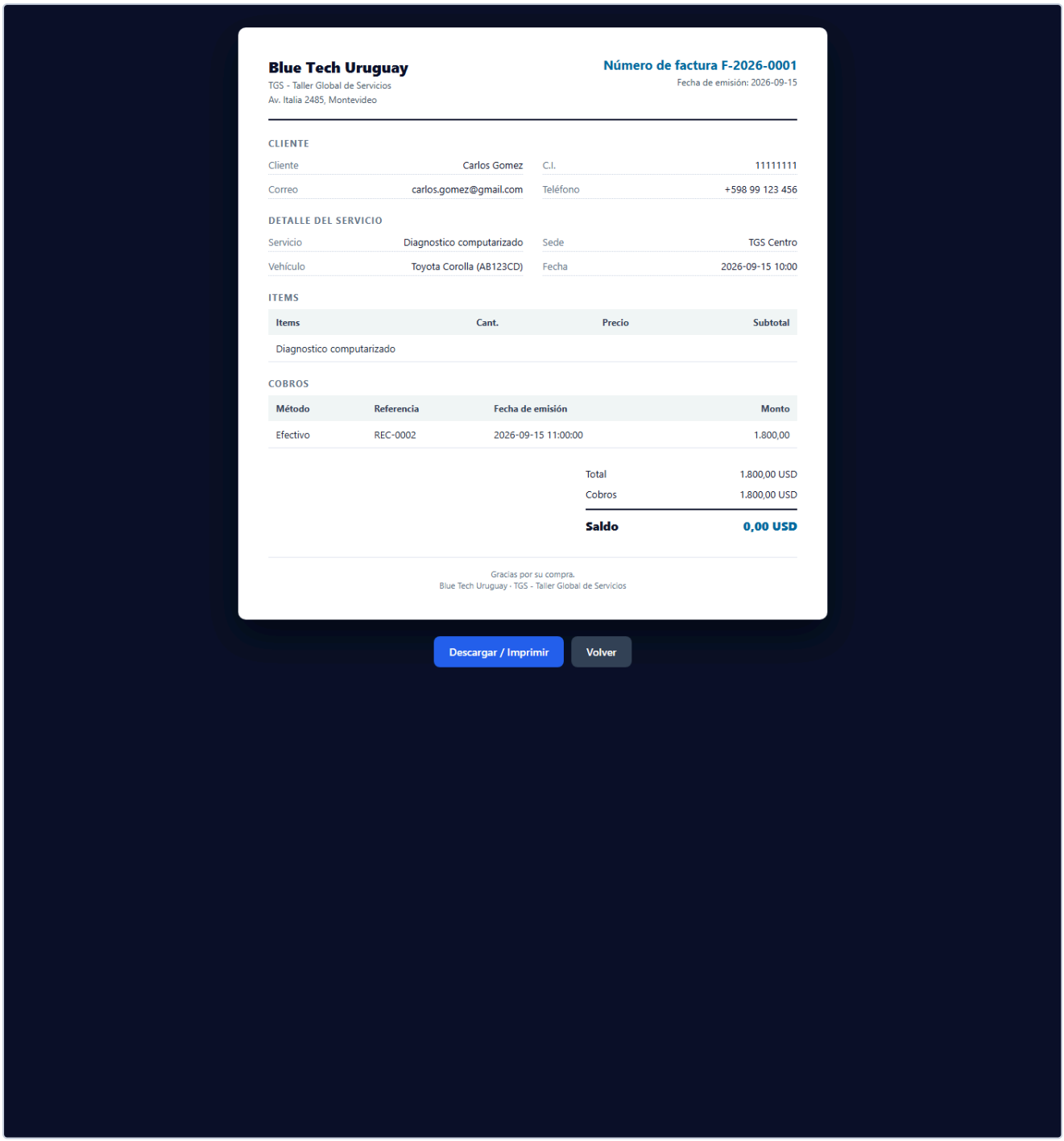


Figura 9. Factura electrónica (`factura.php`): comprobante numerado y descargable en PDF que se emite al validar el pago.

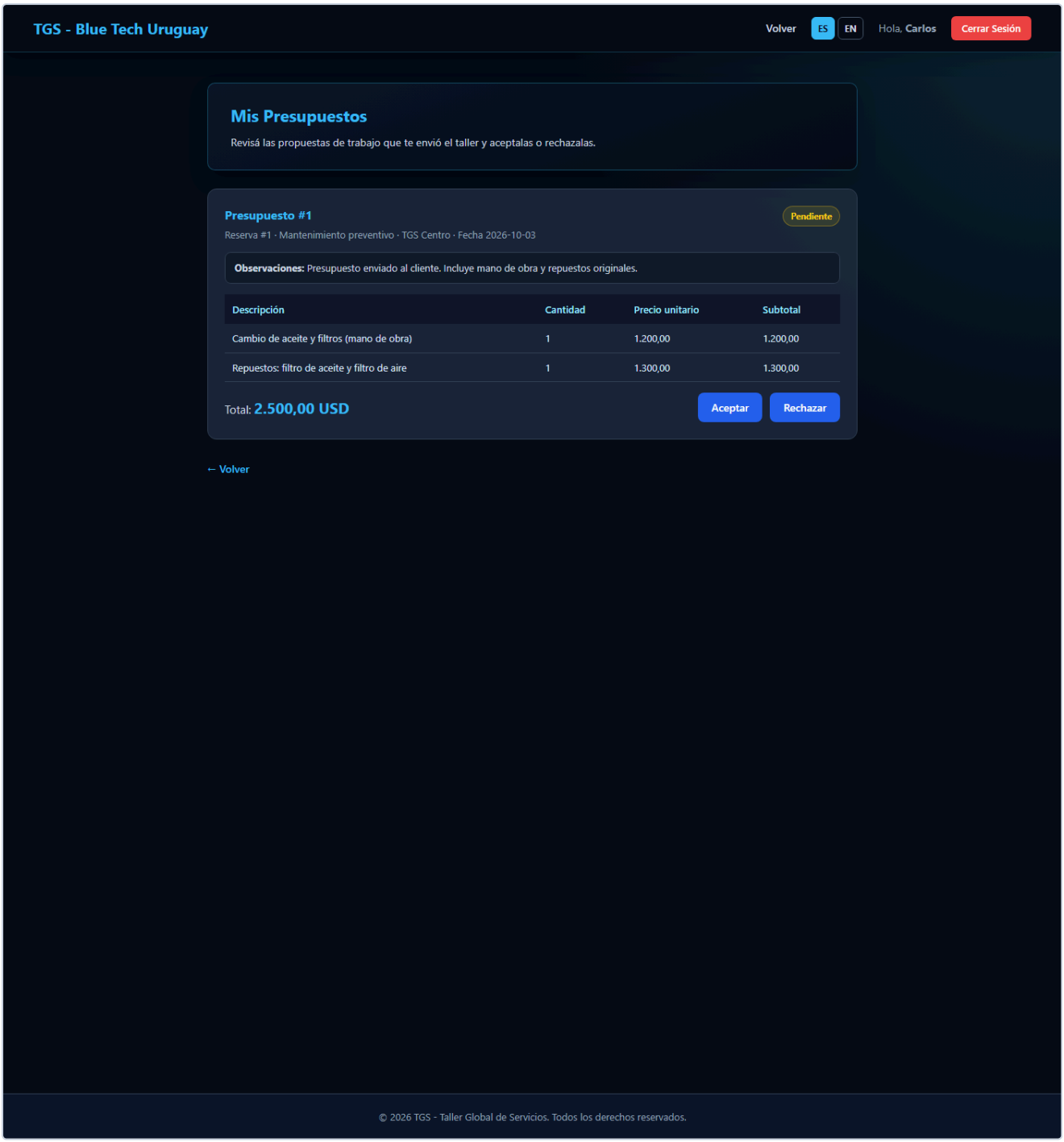


Figura 10. Presupuestos del cliente (`presupuestos.php`): consulta del presupuesto con sus ítems y total, y la opción de aceptarlo o rechazarlo.

7.7 Notificaciones por correo

El módulo de avisos registra siempre la notificación en la base (tabla `notificacion`) y la envía solo si hay transporte de correo configurado; si no lo hay, queda en estado `pendiente` y se puede reenviar después. El cliente SMTP propio implementa el diálogo completo (EHLO, autenticación, MAIL FROM, RCPT TO, DATA) con soporte de TLS y de `STARTTLS` .

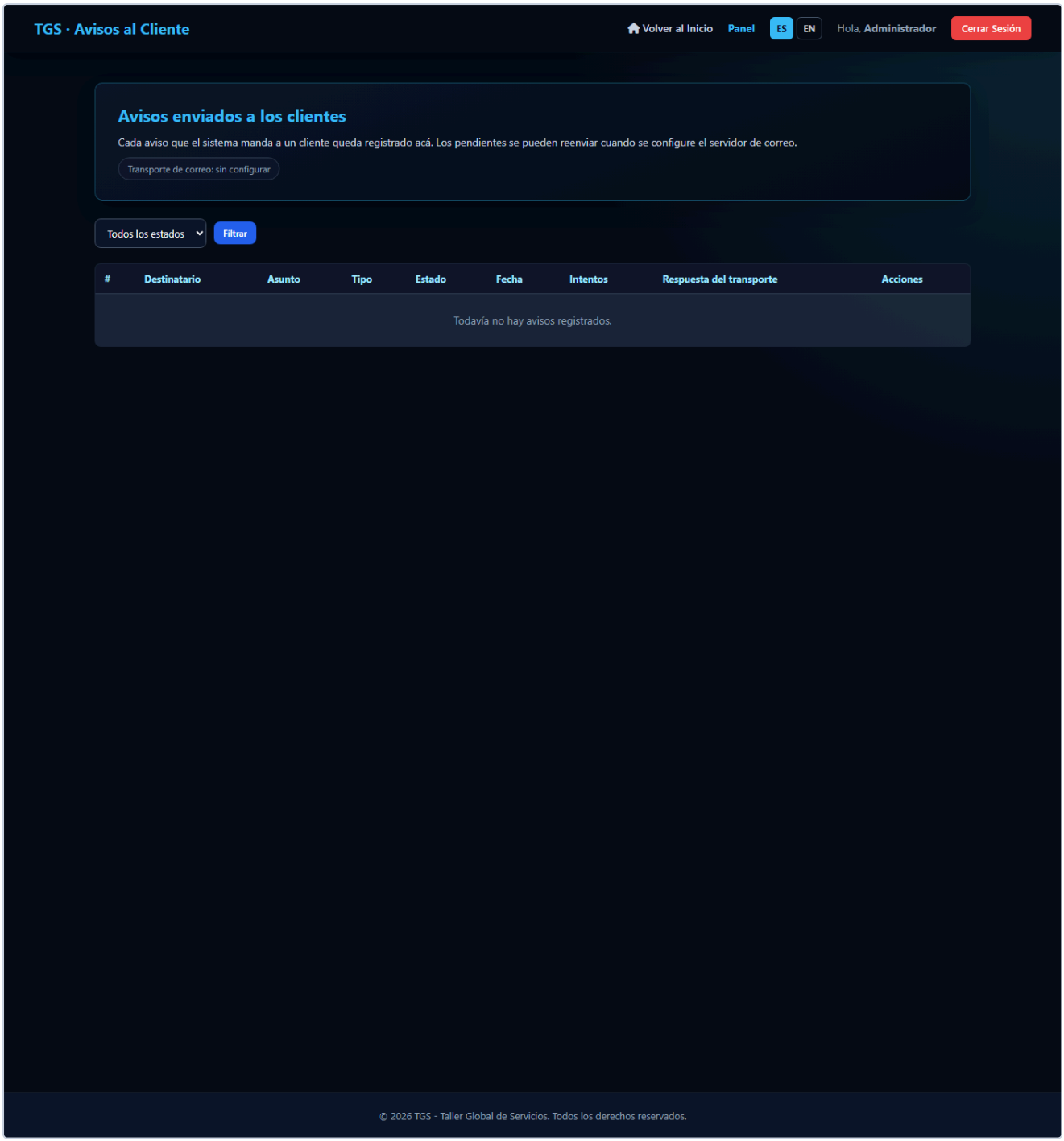


Figura 11. Cola de avisos (`notificaciones.php`): mensajes enviados al cliente, su tipo, el estado del envío, el número de intentos y la opción de reenviar.

7.8 Gestión administrativa



Figura 12. Matriz de permisos (`permisos.php`): 29 permisos por rol, editables en caliente. Los permisos críticos del administrador están protegidos para que el sistema nunca quede sin gestión.

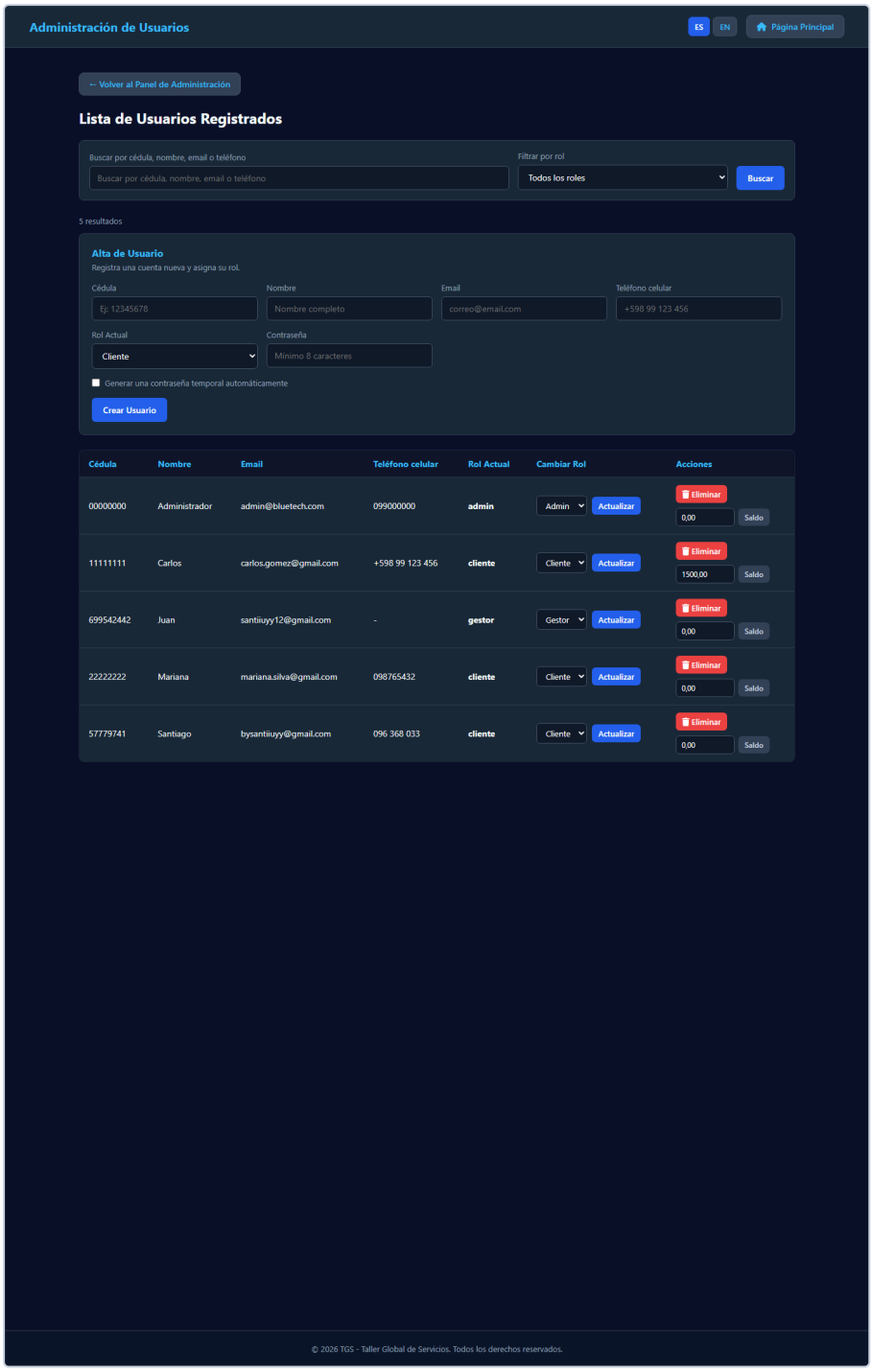


Figura 13. Administración de usuarios (`administrar_usuarios.php`): alta, edición, baja y filtros por rol y por texto.

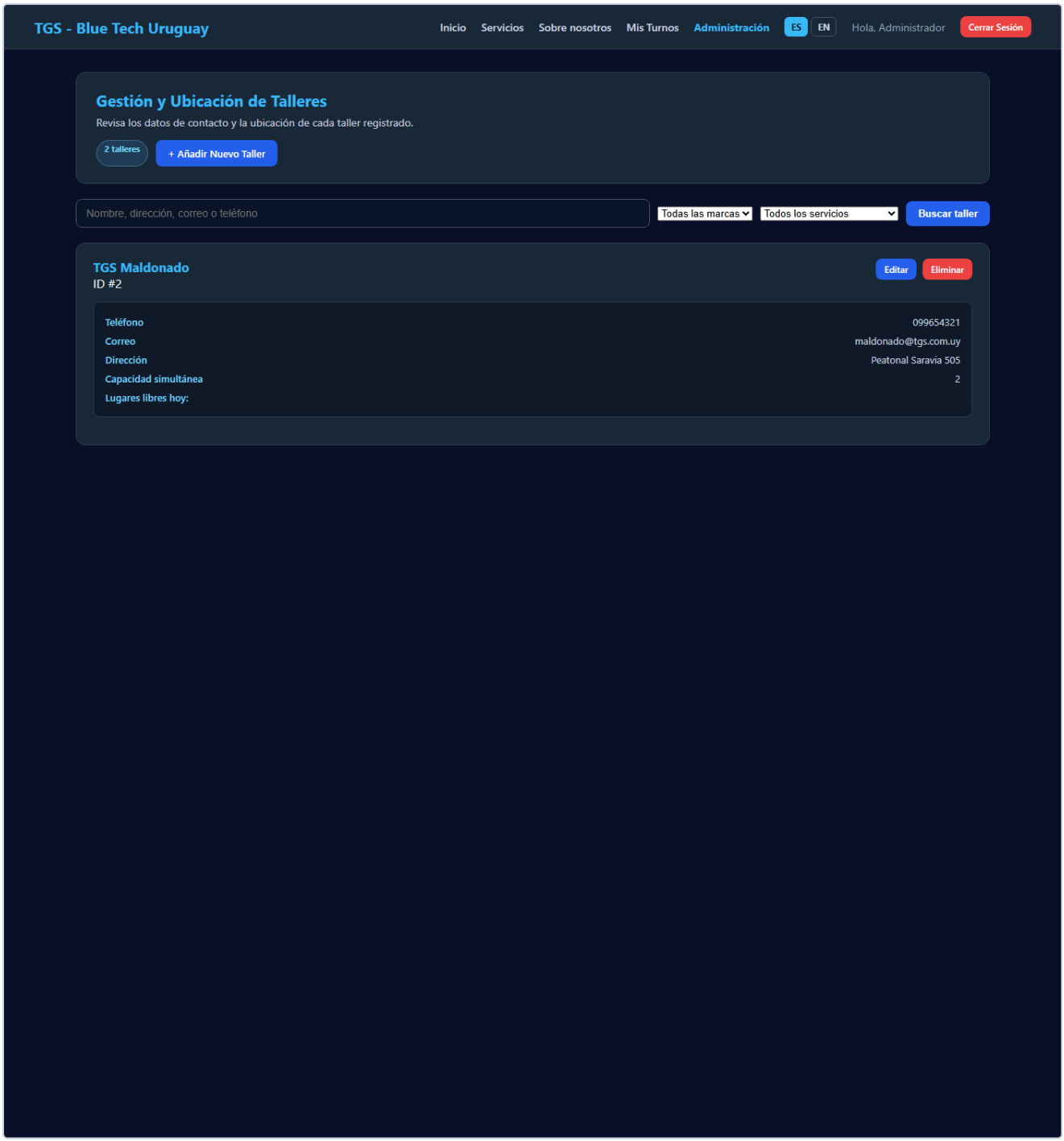


Figura 14. Administración de sedes (`administrar_talleres.php`): tarjetas con datos, horario, filtros por texto, marca y tipo de servicio, y enlace a la edición y al alta.

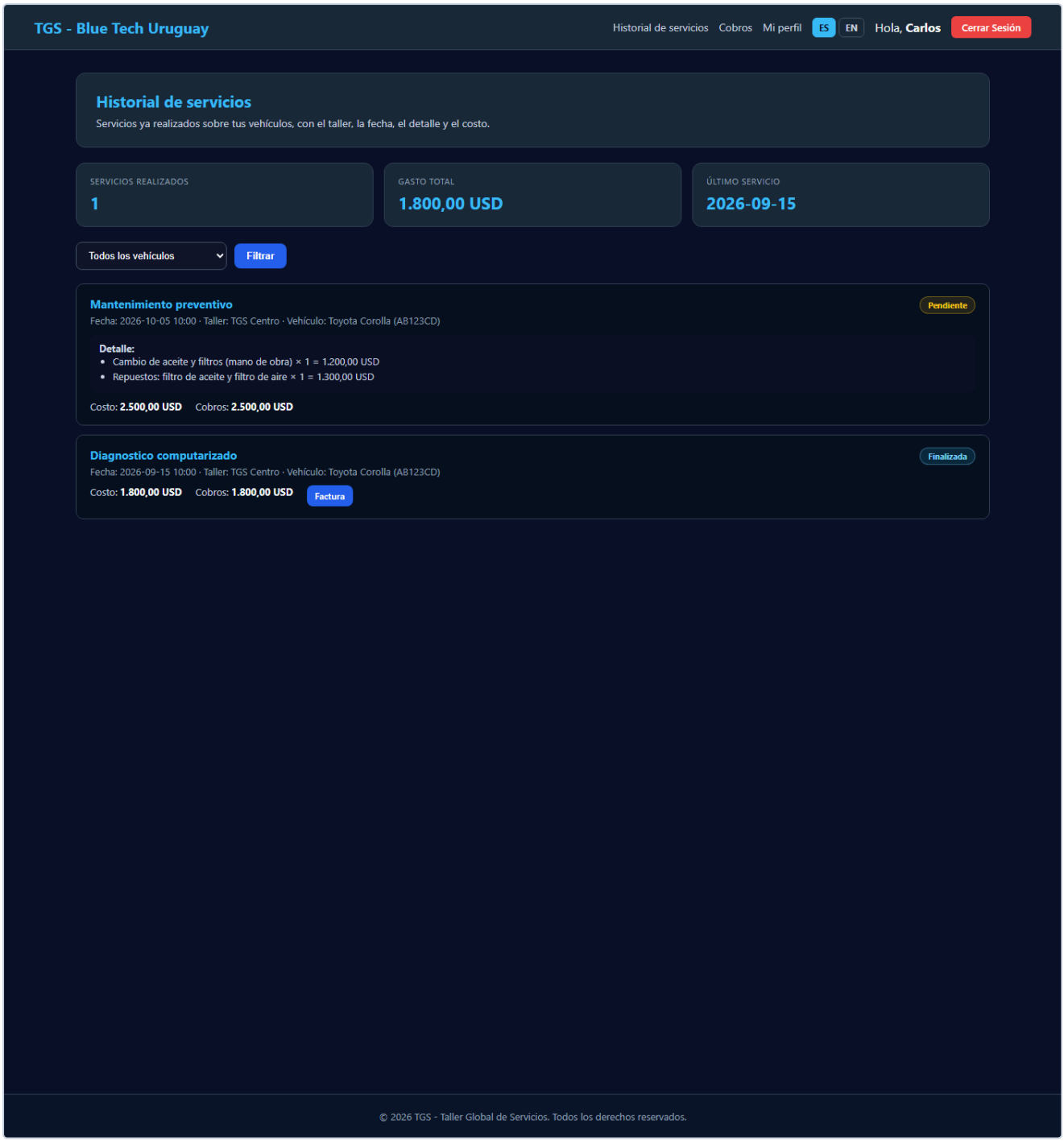


Figura 15. Historial de servicios del cliente (`historial_cliente.php`): por vehículo, con sede, fecha, detalle, costo, cobros y el enlace a la factura.

8. Seguridad de la información

La seguridad no es un agregado del final: es una condición de diseño transversal, presente en cada una de las 42 pantallas. Este capítulo resume los controles implementados, la evidencia de cada uno y el resultado de su verificación.

8.1 Autenticación y gestión de sesión

- **Almacenamiento de claves:** se usa `password_hash()` con bcrypt (`PASSWORD_DEFAULT`) y la verificación con `password_verify()`. Las claves nunca se guardan ni se comparan en texto plano.
- **Cookie de sesión:** `httponly` (no accesible desde JavaScript), `SameSite=Lax` (corta el envío de cookies en peticiones de terceros) y `secure` automático si lasitio se sirve por HTTPS.
- **Regeneración del identificador:** tras autenticarse se llama a `session_regenerate_id(true)`, lo que evita la fijación de sesión.
- **Error genérico:** el mensaje de login no distingue entre correo inexistente y clave incorrecta, para no permitir enumeración de usuarios.

8.2 Protección contra CSRF

Todos los formularios `POST` incluyen un token oculto y son validados en el servidor antes de ejecutar la acción. La suite `test_csrf.sh` verifica, entre otros casos, que un POST sin token válido no modifica nada, que el token de otra sesión se rechaza, y que el token se consume de forma coherente.

8.3 Prevención de inyección SQL

Todas las entradas del usuario llegan a la base como parámetro de una consulta preparada. Los identificadores que necesitan concatenarse se castean antes (`intval()`, `FILTER_VALIDATE_INT`). Durante la revisión del proyecto se detectó que cuatro páginas (portada, alta y edición de sede, gestión de sedes) ainda usaban la API antigua `mysqli` con SQL construido por concatenación; se migraron a PDO, con lo que el proyecto queda con **una única vía de conexión** y sin SQL construido a mano en ningún punto del código.

8.4 Control de acceso por permisos

La matriz de permisos declara 29 permisos para tres roles. El código define los valores por defecto y la tabla `permiso_rol` guarda las modificaciones que haga el administrador desde `permisos.php`: si no hay fila para un par (rol, permiso), manda la matriz del código; si la hay, manda la tabla. Los tres permisos críticos (`ver_panel_admin`, `ver_matriz_permisos`, `editar_matriz_permisos`) no se pueden deshabilitar, para que el sistema nunca quede sin administración.

Además, cada página comprueba la **propiedad** del recurso, no solo el permiso: un cliente no puede ver ni modificar la reserva de otro cliente porque la consulta filtra por la cédula de la sesión (prueba `test_ownership.sh`).

8.5 Manejo de errores y exposición de información

El contenedor usa `php.ini-production` con `display_errors=Off` y `log_errors=On`; el detalle real queda en el log del contenedor y el visitante ve un mensaje genérico. Se corrigió además el caso en que las páginas se ejecutaban con `display_errors=On` durante las primeras entregas, lo que exponía detalles internos en pantalla.

8.6 Protección de los datos de la empresa y de la base

Los datos bancarios que ve el cliente (alias, CBU y QR) se cargan desde el panel del administrador y se muestran solo si están activos; ningún cliente puede crearlos ni modificarlos. Las credenciales de la base de datos no están en el código: viven en `.env` (plantilla versionada) y `.env.local` (valores locales, ignorado por Git).

Además, el acceso a MySQL está segmentado por roles (capítulo 6.4.1): la aplicación usa el usuario `tgs`, los informes y la auditoría usan `tgs_lectura`, que solo puede consultar, y la administración de la base usa `tgs_admin`. Así, si un usuario de solo lectura se compromete, no puede modificar los datos del negocio.

8.7 Evidencia de la verificación

Control	Evidencia
CSRF en todos los POST	`test_csrf.sh` en verde; auditoría que recorre todos los formularios del código
Contraseñas con hash	Verificación directa en la base: la columna `password` almacena hashes `$2y$10$`
Consultas preparadas	Auditoría de código sin SQL concatenado; migración de las cuatro páginas que aún usaban `mysqli`
Control de acceso	`test_permisos.sh`, `test_matriz.sh`, `test_ownership.sh` y `crawl_403.sh` (recorre todas las páginas de cada rol y verifica que ningún enlace devuelva 403)
Acceso segmentado a la base de datos	El rol `tgs_lectura` puede consultar y MySQL le rechaza cualquier escritura: la comprobación es automática dentro de `crear_roles_bd.sh`
Sin errores PHP visibles	Auditoría de las 42 páginas con `display_errors=On`: 0 avisos y 0 errores
Integridad de la base	Sin claves foráneas rotas ni filas huérfanas; tablas de unión coherentes
Archivos del proyecto no expuestos	El `.htaccess` bloquea `.env`, `.env.local`, `.git/` y las carpetas `db/`, `docs/`, `scripts/` y `backups/`; la comprobación se hace con peticiones reales contra el sitio publicado

8.8 Publicación y control del acceso

El sitio se publica con **Cloudflare Tunnel** (capítulo 11.7), de modo que queda disponible desde internet sin abrir puertos en el router. Al estar expuesto, el proyecto suma dos controles específicos:

- **Claves de demostración que no se publican:** en el repositorio solo están los hash bcrypt; las contraseñas se entregan por separado. La suite de pruebas las toma de un archivo local (`tests/credenciales.local.sh`) que queda fuera del control de versiones.
- **Registro controlado:** el registro de cuentas se puede cerrar (`REGISTRO_ABIERTO=0`) o proteger con un código (`REGISTRO_CODIGO`), sin tocar el código. Así el requisito de registro del enunciado se cumple y, a la vez, nadie puede darse de alta por su cuenta mientras el sitio esté abierto al público.

9. Plan y resultados de las pruebas

9.1 Plan de pruebas

El plan de pruebas define qué se verifica, con qué técnica y cuándo se da por' aprobado. Se combinan las dos familias que pide el enunciado.

Nivel	Tipo (caja)	Técnica	Alcance	Criterio de aprobación
Unitarias y estructurales	Blanca	Revisión del código y linter de sintaxis	42 archivos PHP	0 errores de sintaxis y 0 avisos en la auditoría
Integración	Mixta	Ejecución sobre la aplicación real en Docker, con base de datos real	Los 40 requisitos y las 5 brechas G1–G5	Cada caso devuelve el código y el contenido esperados
Sistema	Negra	Recorrido completo por rol con capturas de las 24 pantallas	Cliente, gestor y administrador	Recorrido sin errores y con los datos correctos
Regresión	Mixta	Suite completa antes de cada entrega	Todo el proyecto	28/28 suites en verde y 0 casos fallidos
Seguridad	Negra y blanca	CSRF, SQLi, control de acceso, enumeración de usuarios, cookies	Formularios y páginas restringidas	0 vulnerabilidades abiertas

Cada prueba se ejecuta sobre el sistema desplegado en Docker, no sobre móquinas simuladas: el script de pruebas inicia sesión con las credenciales reales, obtiene el token CSRF de cada página y comprueba el resultado. Antes de cada suite se restauran los datos de demostración, de modo que las pruebas no se pisan entre sí.

9.2 Resultados obtenidos

Suite	Qué verifica	Resultado
`test_csrf.sh`	Token CSRF en todos los formularios POST, inclusión de la maniobra con token de otra sesión	**Correcto**
`test_alta.sh`	Alta de usuarios, validación de correo duplicado, roles inválidos, clave corta y clave automática	**Correcto**
`test_editar.sh`	Edición de vehículo con datos válidos y con Attempts de inyección	**Correcto**
`test_perfil.sh`	Edición de perfil, cambio de clave y validación de la clave actual	**Correcto**
`test_permisos.sh`	Acceso de cada rol a cada página, redirección de intrusos y acceso sin sesión	**Correcto**
`test_disp.sh`	Reserva de turno, día cerrado, hora fuera de horario y franja completa	**Correcto**
`test_repro.sh`	Reprogramación válida e inválida, cancelación y actualización del histórico	**Correcto**
`test_alta_taller.sh`	Alta de sede con horario, marcas y servicios, y reserva sobre la sede nueva	**Correcto**
`test_smtp.sh`	Registro de la notificación al finalizar un servicio	**Correcto**
`test_pagos.sh` / `test_pagos2.sh`	Registro, aprobación, rechazo y anulación de cobros; recálculo del estado de la reserva	**Correcto**
`test_presupuestos.sh`	Alta de presupuesto con ítems, totales, aceptación y rechazo por parte del cliente	**Correcto**
`test_notificaciones.sh`	Cola de avisos, estados de envío y reenvío	**Correcto**
`test_matriz.sh`	Edición de la matriz de permisos y efecto inmediato en el acceso	**Correcto**
`test_baja.sh`	Baja de cuenta de cliente y de usuario del sistema	**Correcto**
`test_panel.sh` / `test_nav.sh`	Contenido de los tres paneles y navegación de cada rol	**Correcto**
`test_agenda.sh` / `test_detalle.sh` / `test_modal.sh`	Agenda del gestor, detalle del turno y edición rápida	**Correcto**
`test_filtros.sh` / `test_filtros2.sh` / `test_filtros3.sh`	Filtros por texto, rol, marca y servicio, con y sin resultados y con intento de inyección	**Correcto**
`test_ownership.sh`	Un cliente no puede ver ni modificar datos de otro	**Correcto**
`test_pago_cliente.sh` / `test_a_pagar.sh`	Pago con saldo prepago, reporte de pago y cuentas pendientes	**Correcto**
`test_idiomas.sh`	Bilingüismo: traducción en las 24 pantallas, persistencia del idioma y selector en todas	**Correcto**
`test_gaps.sh` (20 casos)	Verificación de las brechas G1–G5 del enunciado	**Correcto**

> **Resultado global de la última ejecución completa:** 28 de 28 suites en verde, 20 de 20 casos de brechas en verde, 0 errores de sintaxis en los 42 archivos PHP y 0 filas huérfanas en la base de datos.
>
> **La suite forma parte del proyecto.** Los 38 scripts están versionados en la carpeta `tests/` del repositorio, con un `README.md` que explica qué verifica cada uno y cómo ejecutarlos. Se ejecutan contra la aplicación real que corre en Docker, dentro de la máquina virtual del proyecto. Las claves de las cuentas de demostración se leen de un archivo local que **no** se versiona (`tests/credenciales.local.sh`), porque el sitio queda disponible en internet durante la defensa.

9.3 Correcciones Derivadas de las pruebas

Las pruebas no solo verifican: también обнаружи detectaron defectos reales que se corrigieron durante el proyecto. Estos son algunos de los más relevantes para la defensa, porque explican decisiones de diseño:

Defecto detectado	Causa	Solución aplicada
Error 500 al editar una sede	`bind_param()` de `mysqli` exige variables y no admite expresiones: se pasaba `(int) $taller['id_taller']`	Migración completa a PDO, con parámetros separados de la consulta
Error 500 al listar las sedes	Se pasaba una conexión `mysqli` a una función que exige `PDO`	Misma migración; el proyecto queda con una sola API de acceso a datos
Acceso denegado al abrir «Mis pagos» desde el perfil	El enlace apuntaba siempre a la pantalla del cliente	Enlace según rol: el cliente ve sus pagos y el taller, la administración de cobros
Botón duplicado en el panel del administrador	Bloque de código agregado dos veces	Se eliminó el repetido y se agregó una verificación de enlaces por rol
Filtro de marca que no filtraba nada	La marca se enlazaba como número y MySQL comparaba «Toyota» con 0	Enlace por tipo correcto y prueba específica con dos marcas
Errores PHP visibles al visitante	El contenedor arrancaba con `display_errors=On`	`php.ini-production` y auditoría de las 42 páginas

10. Trabajo en equipo y control de versiones

10.1 Composición y roles del equipo

Integrante	C.I.	Rol en el proyecto	Responsabilidades principales
Santiago Figueroa	5.777.974-1	Coordinación y backend	Coordinación del equipo, arquitectura, modelo de datos, módulo de reservas y disponibilidad, integración de la solución
Alex Torres	6.747.915-7	Front-end e interfaz	Diseno de la interfaz, hojas de estilo, navegación, bilingüismo y accesibilidad
Facundo Fernandez	5.717.364-8	Base de datos	Modelo relacional, DDL, datos de prueba, consultas y reportes, diccionario de datos
Mariano Di Trani	5.743.443-6	Calidad y pruebas	Suite automatizada, plan de pruebas, auditoría de seguridad y de código
Alejo Mussetti	5.750.437-6	Infraestructura y ASO	Despliegue en Docker, scripts de operación, respaldos, instalación de la máquina virtual

10.2 Control de versiones

El proyecto se versiona con Git en un repositorio de GitHub. Cada cambio corresponde a un commit con un mensaje descriptivo que explica el *qué* y no solo el *cómo*. El histórico permite reconstruir el proyecto paso a paso, que es justamente lo que se evalúa en la defensa oral.

```
git log --oneline
9e0fa67 Evitar el acceso denegado al abrir Mis pagos desde el perfil
c1fb1ee Eliminar el boton duplicado de datos de pago en el panel admin
039d19d Aclarar el panel de datos bancarios: listado arriba, formulario abajo
c85c7c4 Cerrar los 5 requisitos que faltaban del enunciado
2ad2bd2 Agregar guia de defensa y entregables tecnicos de la 2a entrega
37e6af8 Ocultar los errores de PHP al visitante y registrarlos en el log
78372be Unificar la interfaz de los paneles con el diseno TGS
69bb73f Corregir el cambio de idioma entre espanol e ingles
d674559 Permitir que el cliente declare su pago y vea los datos bancarios
032e8a5 Exigir token CSRF en todos los formularios POST
566cc99 Actualizar el README con los modulos implementados
c714080 Agregar modulos de gestion: pagos, presupuestos, permisos, agenda y disponibilidad
7c16763 Migrar a Docker, externalizar configuracion y comentar el codigo PHP
5290285 Actualizar sistema general
```

10.3 Cronograma del proyecto

Etapa	Periodo	Entregable
Análisis y requisitos	Mayo 2026	Documento de requisitos, lectura del enunciado, acta de integración y conformación del grupo
Modelado y diseño	Junio 2026	MER, diccionario de datos, diagramas UML, glosario bilingüe
Primera entrega	6 de julio de 2026	Versión funcional del módulo de reservas y presentación oral
Ampliación funcional	Julio – agosto 2026	Pagos, presupuestos, permisos, agenda, disponibilidad y datos bancarios
Segunda entrega	14 de setiembre de 2026	Documentación técnica, entregables de ingeniería de software, base de datos completa y despliegue documentado
Estabilización y pruebas	Setiembre – octubre 2026	Cierre de brechas G1–G5, suite de pruebas completa, auditoría de seguridad e inglés
Tercera entrega y defensa	Octubre 2026	Presentación del software completo y entrega de la máquina virtual

10.4 Riesgos identificados y tratamiento

Riesgo	Probabilidad / impacto	Tratamiento aplicado
Desviación del alcance (el enunciado exige 40 funcionalidades)	Alta / Alto	Traza de requisitos contra el código y revisión de brechas: se detectaron cinco y se cerraron (G1–G5)
Pérdida de la base de datos	Baja / Crítico	Script de respaldo, rotación de los cinco respaldos más recientes, tarea programada diaria y restauración probada en clase
Caída del servidor durante la defensa	Media / Crítico	Script `caida.sh` que simula la caída y `restaurar.sh` que deja el sitio en pie; se demuestra en vivo
Agotamiento del tiempo antes de la defensa	Alta / Medio	Planificación por sprints, definición de «hecho» y tareas nuevas solo si cierran un requisito del enunciado
Errores de seguridad detectados	Media / Crítico	Suite automatizada que corre antes de cada entrega y auditoría de las 42 páginas
Diferencias entre máquinas del equipo	Media / Medio	Configuración externalizada (`.env`) e instalación con un solo comando

10.5 Métricas del proyecto

Métrica	Valor	Comentario
Pantallas implementadas	42 archivos PHP de vista	24 de ellas con captura en este documento
Líneas de código PHP	12.755	Con comentarios y código de estilos
Tablas de la base de datos	16	Normalizadas hasta 3FN
Permisos de la matriz	29 × 3 roles	Editables en caliente
Pruebas automatizadas	28 suites + 20 casos	100 % en verde en la última ejecución
Commits	18	Todos con mensaje descriptivo
Cobertura de los requisitos	40 de 40	Verificado con la matriz de trazabilidad (capítulo 4)
Errores de sintaxis / warnings	0 / 0	Auditoría sobre las 42 páginas

11. Infraestructura y administración de sistemas operativos

El enunciado de la cátedra de Administración de Sistemas Operativos pide, para la tercera entrega, un servidor web y otro de respaldo en funcionamiento, la seguridad de los servidores, un script operador de centro de cómputos y la entrega de una máquina virtual en formato `.OVA` con sus scripts. Este capítulo describe lo construido.

11.1 Máquina virtual y contenedores

Elemento	Configuración
Máquina virtual	VirtualBox con Ubuntu 26.04 LTS, 2 vCPU, 4 GB de RAM y 40 GB de disco; dirección IP de la red local con la Web published en el puerto 8080
Contenedor web	`tgs_app` sobre la imagen oficial `php:8.4-apache`, con el proyecto montado en caliente (los cambios se ven sin reconstruir la imagen) y la configuración de PHP en modo producción
Contenedor de datos	`tgs_db` con `mysql:8.4`, *healthcheck* cada 10 segundos y política de reinicio automático
Persistencia	Volumen Docker `tgs_db_data`: la base sobrevive a la reinstalación de los contenedores
Red	Red propia del proyecto, con comunicación interna por nombre de servicio (`db`) y publicación del puerto 8080 en el host
Configuración	Variables de entorno: `.env` (plantilla versionada) y `.env.local` (valores de la máquina, ignorado por Git)

11.2 Scripts de operación

La administración del sistema se resuelve con siete scripts en `scripts/`, todos idempotentes y con mensajes claros para el operador:

Script	Qué hace
`instalar.sh`	Prepara el entorno: genera `.env.local` si falta, levanta los contenedores, espera a que MySQL esté listo, carga el esquema y los datos de prueba y verifica la instalación
`respaldar.sh`	Genera un `mysqldump` con la opción `--single-transaction`, lo guarda en `backups/` con fecha y hora, lo protege con permisos restrictivos y rota los respaldos antiguos conservando los cinco más recientes
`restaurar.sh`	Restaura un respaldo elegido: detiene los contenedores, borra el volumen, lo recrea, importa el `.sql` y vuelve a levantar el sistema
`caida.sh`	Simula la caída total del servidor eliminando el volumen de datos, para demostrar en la defensa la política de recuperación
`estado.sh`	Muestra el estado de los contenedores, el uso de disco, la última hora de los logs y la cantidad de respaldos disponibles
`operador.sh`	**Script operador de centro de cómputos**: reúne en un solo comando las tareas habituales de administración (servicios, red, procesos, respaldos, restauración, usuarios y grupos, orígenes de datos y acceso a los registros). Se abre con un menú interactivo o se llama por tarea: `bash scripts/operador.sh estado`
`crear_roles_bd.sh`	Crea los tres roles de usuario de MySQL (aplicación, solo lectura y administración) y comprueba que el rol de lectura efectivamente no puede escribir
`firewall.sh`	Configura el firewall del servidor con `iptables` (entrada en `DROP`, SSH y sitio solo desde la red local, puertos de Docker filtrados en `DOCKER-USER`), guarda las reglas para el arranque y las restaura solo si se pierde la conexión
`tunel.sh`, `tunel_nombrado.sh`, `tunel_vigilante.sh`	Publicación del sitio: túnel temporal, túnel con nombre permanente y vigilante que mantiene publicada la URL vigente

El script operador es el entregable que la cátedra pide como «primera versión del script de operador de centro de cómputos». Sus tareas son:

Tarea	Qué muestra o hace
`servicios`	Estado de Docker, SSH y cron, y las tareas programadas del usuario; permite arrancar, parar o reiniciar cada contenedor
`red`	Interfaces y direcciones, puertos en escucha, tabla de rutas, servidores de nombres, redes de Docker y la IP pública de acceso
`procesos`	Carga del sistema, memoria, uso de disco y los procesos con mayor consumo de CPU y de memoria
`respaldos`	Lista los respaldos disponibles con fecha y tamaño, e indica el comando para generar uno nuevo
`restauracion`	Restaura un respaldo pedido por nombre, con confirmación interactiva antes de tocar los datos
`usuarios`	Usuarios del sistema y de MySQL, con los permisos de cada rol; permite crear usuarios del sistema
`grupos`	Grupos del sistema y pertenencia de los usuarios; permite agregar o quitar usuarios de un grupo
`origenes`	Bases, tablas y vistas de MySQL; permite crear bases y usuarios nuevos y copiar una base completa
`registros`	Acceso a los registros: logs del servidor web y de la base, log de la tarea programada y auditoría de cambios en las reservas y en los avisos al cliente
`estado`	Resumen general del servidor en una sola pantalla

Las tareas que necesitan privilegios de administrador (crear usuarios o grupos del sistema) detectan que no hay `sudo` disponible y lo explican, en lugar de fallar: el resto del operador sigue siendo utilizable sin privilegios.

11.3 Política de respaldo y recuperación

> **Tarea programada.** El respaldo se automatizó con una entrada de `crontab` del usuario del servidor, que ejecuta `scripts/respaldar.sh` todos los días a las 3:17 y deja el log en `backups/cron.log`:
>
> ```
> 17 3 * * * /bin/bash /home/taller/Desktop/tgs/scripts/respaldar.sh \
>             >> /home/taller/Desktop/tgs/backups/cron.log 2>&1
> ```
>
> La rotaciónautomática (conservar los cinco respaldos más recientes) evita que el disco se llene, y la restauración se probó en la defensa con el guion del video de caída.

11.4 Seguridad de los servidores

La máquina virtual aplica las medidas básicas de un servidor de centro de cómputos: gestión de la base de datos con un usuario de aplicación (`tgs`) sin permisos administrativos (el `root` se usa solo para los respaldos), actualizaciones de seguridad automáticas mediante `unattended-upgrades`, y el **firewall con `iptables`** que documenta el script `scripts/firewall.sh`.

Regla	Propósito
Política por defecto de entrada	`DROP`: se descarta todo lo que llega desde afuera
Tráfico ya establecido y bucle local	Se acepta (no se corta una conexión en curso)
Eco de ICMP (ping)	Se acepta, para poder comprobar que el servidor responde
Acceso por SSH (22) y al sitio (8080)	Solo desde la red local `192.168.1.0/24`
Redes de Docker	Se acepta el tráfico de los contenedores (web y base de datos)
Puertos publicados por Docker	Se filtran en la cadena `DOCKER-USER`, que es la única que Docker no administra: el 8080 publicado queda restringido a la red local

El script se puede ejecutar cuantas veces sea necesario: aplica las reglas, las guarda en `/etc/iptables/rules.v4` y habilita el servicio `tgs-firewall.service`, que las restaura al arrancar el servidor. Además tiene un **temporizador de seguridad**: si la conexión se corta durante la aplicación, el propio script restaura las reglas anteriores, de modo que nunca se pierde el acceso al servidor.

El túnel de publicación corre como servicio del sistema (`tgs-tunel.service`), con reinicio automático: si el proceso se cae, systemd lo vuelve a levantar y el vigilante publica la URL vigente en `url-publica.txt`.

11.5 Entrega de la máquina virtual

La entrega de la tercera entrega es la máquina virtual en formato `.OVA`, configurada y con los scripts dentro. El procedimiento de exportación y las credenciales del administrador se documentan en el Anexo

C, junto con la verificación de que la imagen arranca, levanta los contenedores y sirve el sitio correctamente.

11.6 Manual de instalación (resumen operativo)

```
# 1. Requisitos: VirtualBox, Docker y Docker Compose en la máquina
# 2. Copiar el proyecto en /home/taller/Desktop/tgs
cd /home/taller/Desktop/tgs
# 3. Instalar (crea .env.local, levanta contenedores y carga la base)
bash scripts/instalar.sh
# 4. Comprobar el estado
bash scripts/estado.sh
# 5. Respaldar y, si hace falta, restaurar
bash scripts/respaldar.sh
bash scripts/restaurar.sh tgs_AAAAMMDD_HHMMSS.sql
# 6. Simular una caída y recuperar el servicio
bash scripts/caida.sh
bash scripts/restaurar.sh tgs_AAAAMMDD_HHMMSS.sql
```

11.7 Publicación del sitio en internet

La máquina virtual del proyecto vive en una red doméstica, detrás del router y sin dirección IP pública. Para presentar el sistema fuera de esa red (por ejemplo, desde el aula de la UNT en la defensa) se usa **Cloudflare Tunnel**: el servidor se conecta hacia afuera a Cloudflare y Cloudflare reenvía el tráfico a la aplicación. No hay que abrir puertos ni tocar la configuración del router, y el sitio queda con HTTPS (certificado automático).

Script / acción	Qué hace
`bash scripts/tunel.sh iniciar`	Levanta el túnel y publica la URL pública en la salida estándar
`bash scripts/tunel.sh estado`	Muestra la URL activa, comprueba que el sitio responda desde internet y consulta el estado del proceso
`bash scripts/tunel.sh detener`	Corta el túnel y deja de publicar el sitio
`bash scripts/tunel.sh instalar`	Instala `cloudflared` en el usuario, sin permisos de administrador

La URL que entrega el modo temporal tiene la forma `https://<palabras>.trycloudflare.com` y cambia cada vez que se reinicia el túnel. Para un **nombre permanente** hay dos caminos libres:

- **Cloudflare con un dominio propio**: se crea una cuenta gratuita en Cloudflare, se apunta el dominio a sus servidores y se declara un túnel con nombre; el sitio queda en `https://tgs.<tu-dominio>` con certificado automático.
- **Subdominio gratis con DuckDNS o FreeDNS**: servicios que dan un subdominio gratuito (por ejemplo `tgs.duckdns.org`) que se actualiza desde el propio servidor con una tarea programada. El subdominio se configura como nombre público del túnel de Cloudflare.

> **Seguridad previa a publicar.** Antes de exponer el sitio se corrigió una falla real: Apache servía todos los archivos de la carpeta del proyecto, de modo que `/.env.local` entregaba la clave de root de MySQL y `/.git/` el historial completo del repositorio. El archivo `.htaccess` del proyecto bloquea ahora los archivos ocultos, el historial de Git y las carpetas sensibles (`db/`, `docs/`, `scripts/`, `backups/`), y agrega cabeceras de seguridad básicas. La verificación se hace con peticiones reales contra el sitio publicado, no solo de forma local.

El sitio publicado mantiene el mismo acceso del sistema interno: la sesión, los permisos por rol y la matriz de permisos siguen siendo los que se describen en el capítulo 8. Para una demostración pública conviene cambiar las claves de las cuentas de prueba y restringir el registro abierto.

12. Glosario bilingüe

La interfaz esta disponible en español e inglés. La tabla siguiente es el glosario de la interfaz, tiles que se puede entregar en la parte de Inglés del proyecto.

Español	English	Dónde aparece
Taller Global de Servicios	Global Workshop Services	Marca del proyecto
Iniciar Sesión / Ingresar	Sign In / Log In	`login.php`
Registrarse	Create Account	`registro.php`
Panel del Cliente	Customer Panel	`panel_cliente.php`
Panel del Gestor	Manager Panel	`dashboard_gestor.php`
Panel de Administración	Admin Panel	`dashboard_admin.php`
Mis turnos	My appointments	`reservas.php`
Mis vehículos	My vehicles	`panel_cliente.php`
Mis presupuestos	My budgets	`presupuestos.php`
Mis pagos	My payments	`pagos_cliente.php`
Mi perfil	My profile	`perfil.php`
Dar de baja mi cuenta	Delete my account	`eliminar_cuenta.php`
Reservar Turno	Book Appointment	`reservas.php`
Nuevo vehículo	New vehicle	`reservas.php`
Editar vehículo	Edit vehicle	`reservas.php`
Eliminar vehículo	Delete vehicle	`reservas.php`
Editar turno	Edit appointment	`reservas.php`
Cancelar turno	Cancel appointment	`reservas.php`
Sede	Workshop	`reservas.php`, `administrar_talleres.php`
Horario de atención	Opening hours	`administrar_talleres.php`
Capacidad simultánea	Simultaneous capacity	`administrar_talleres.php`
Ubicación en el mapa	Location on the map	`administrar_talleres.php`
Agenda del día	Daily agenda	`agenda_gestor.php`
Franja / horario libre	Free time slot	`agenda_gestor.php`
Turno pendiente	Pending appointment	`agenda_gestor.php`
Estado de cobro	Collection status	`administrar_pagos.php`
Registrar cobro	Register payment	`administrar_pagos.php`
Aprobar / Rechazar	Approve / Reject	`administrar_pagos.php`
Cobros reportados por los clientes	Payments reported by customers	`administrar_pagos.php`
Importe a pagar	Amount to pay	`pagos_cliente.php`
Datos para transferir	Bank details	`pagos_cliente.php`
Alias / CBU	Alias / CBU	`pagos_cliente.php`
Reportar pago	Report payment	`pagos_cliente.php`
Comprobante o referencia	Receipt or reference	`pagos_cliente.php`

Saldo prepago	Prepaid balance	`panel_cliente.php`, `pagos_cliente.php`
Presupuesto	Budget	`administrar_presupuestos.php`
Ítems	Items	`administrar_presupuestos.php`
Aceptar / Rechazar presupuesto	Accept / Reject budget	`presupuestos.php`
Matriz de permisos	Permission matrix	`permisos.php`
Ver / Editar permisos	View / Edit permissions	`permisos.php`
Avisos al cliente	Customer notices	`notificaciones.php`
Reenviar aviso	Send notice again	`notificaciones.php`
Datos bancarios para pagos	Bank details for payments	`datos_pago.php`
Historial de servicios	Service history	`historial_cliente.php`
Factura	Invoice	`factura.php`

13. Conclusiones y trabajo futuro

13.1 Conclusiones

El proyecto cumple el objetivo general: se desarrolló una plataforma web completa e integrada que permite registrarse, reservar servicios mecánicos, planificar los mantenimientos según la disponibilidad real de cada sede, gestionar presupuestos, cobrar y facturar, con herramientas de administración para la empresa.

Los **40 requisitos** del enunciado quedan implementados y verificados, incluidos los cinco puntos que faltaban en la revisión intermedia (factura electrónica, historial por vehículo, vista detallada de sede, filtros por marca y servicio, y pago directo con cuenta prepaga), y cada uno tiene su prueba automatizada asociada.

La **calidad técnica** se apoya en cuatro decisiones que resultaron determinantes: una única conexión PDO con consultas preparadas en todo el proyecto; un modelo de autorización por permisos en lugar de por roles, editable sin tocar el código; la validación de la disponibilidad en el servidor, de modo que el cliente nunca puede forzar un turno inválido; y una suite automatizada que se ejecuta antes de cada entrega y detectó errores reales, varios de los cuales se corrigieron durante el proyecto.

El despliegue en Docker con scripts de operación, respaldo automático y restauración probada hace que la solución sea **reproducible y demostrable**: cualquiera puede instalarla con un comando y, si el servidor cae, recuperarla en minutos durante la propia defensa.

13.2 Trabajo futuro

Mejora	Valor que aporta
Integrar una pasarela de pago real	El pago dejaría de ser un reporte validado por el taller para convertirse en un cobro automático
Aplicación web progresiva (PWA) con notificaciones	Avisos de turno en el celular sin depender del correo
Calendario de disponibilidad por recursos (róbanos, elevadores, box)	Modelo más fino de capacidad, como el de un taller real con varios recursos
Panel de indicadores para el administrador	Facturación, ocupación por sede y tiempos de respuesta, con gráficos
Código QR de la factura y envío por WhatsApp	Verificación rápida del pago y mejor comunicación con el cliente
Respaldo incremental y copia fuera del servidor	Protección ante la pérdida total de la máquina virtual

14. Referencias

Referencias en formato APA 7º edición, como exige el enunciado para la documentación del proyecto.

NORMATIVA E INSTITUCIONAL

- ANEP & Dirección General de Educación Técnico Profesional. (2026). *Proyecto de Pasaje de Grado: Requerimientos de software para plataforma de servicios mecánicos* [Documento institucional]. UTU, Escuela Técnica de Pando.
- ANEP & Dirección General de Educación Técnico Profesional. (2026). *Reglamento del Proyecto de Pasaje de grado* [Documento institucional]. UTU, Escuela Técnica de Pando.

TECNOLOGÍA Y PROGRAMACIÓN

- PHP Documentation Group. (2026). *PHP: Supported versions and documentation*. https://www.php.net/docs/
- The PHP Group. (2026). *PDO — PHP Data Objects*. https://www.php.net/manual/es/book.pdo.php
- Oracle Corporation. (2026). *MySQL 8.4 Reference Manual*. https://dev.mysql.com/doc/refman/8.4/en/
- Docker Inc. (2026). *Docker Compose documentation*. https://docs.docker.com/compose/
- OWASP Foundation. (2025). *OWASP Top 10: Web Application Security Risks*. https://owasp.org/www-project-top-ten/
- OWASP Foundation. (2025). *Cross-Site Request Forgery Prevention Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- NIST. (2023). *NIST Cybersecurity Framework 2.0*. https://www.nist.gov/cyberframework

INGENIERÍA DE SOFTWARE

- Object Management Group. (2017). *Unified Modeling Language (UML) Specification, version 2.6*. OMG.
- Sommerville, I. (2016). *Software engineering* (10.º ed.). Pearson.
- Schach, B.. (2010). *Object-oriented and classical software engineering* (8.º ed.). McGraw-Hill.
- Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9.º ed.). McGraw-Hill.
- Beck, K., Grenning, J., & Sutherland, J. (2020). *The agile manifesto*. https://agilemanifesto.org/
- Schwaber, K., & Sutherland, J. (2020). *The Scrum guide*. https://scrumguides.org/scrum-guide.html

INGLÉS TÉCNICO

- Council of Europe. (2020). *Common European Framework of Reference for Languages: Learning, teaching, assessment — Companion volume*. Council of Europe Publishing.
- Murphy, R. (2019). *English grammar in use* (5.º ed.). Cambridge University Press.

Anexos

Anexo A — Cuentas de demostración

Usuarios de prueba cargados por `db/seed.sql`, con los tres roles del sistema:

Rol	Correo	Qué demuestra
Administrador	admin@bluetech.com	Matriz de permisos, datos bancarios, usuarios, servicios y agenda
Gestor de taller	santiiuyy12@gmail.com	Agenda del día, validación de cobros, presupuestos y sedes
Cliente	carlos.gomez@gmail.com	Turnos, vehículos, historial, pagos con saldo prepago y presupuestos
Clientes adicionales	mariana.silva@gmail.com bysantiiuyy@gmail.com	Verificación de la propiedad de los datos: cada cliente solo ve lo suyo

> **Las contraseñas no se publican en este documento.** Durante la defensa el sitio quedó expuesto en internet, así que las claves de las cuentas de demostración se entregan por separado al docente (por correo o en clase) y no se versionan en el repositorio: en el código fuente quedan únicamente los *hash* bcrypt. La suite de pruebas toma las claves de un archivo local que está fuera del repositorio (`tests/credenciales.local.sh`).
>
> Además, el registro de cuentas nuevas se controla desde el archivo de entorno, sin tocar el código: `REGISTRO_ABIERTO=0` cierra el registro por completo y `REGISTRO_CODIGO` exige un código para darse de alta. Es la forma de tener el sitio disponible para la defensa sin que se llene de cuentas ajenas.

Anexo B — Estructura de archivos del proyecto

```
tgs/
|-- .env                      Variables de entorno globales (plantilla)
|-- .env.local                Variables locales de la maquina (no versionado)
|-- .env.example              Plantilla de configuracion documentada
|-- Dockerfile                Imagen del servidor web
|-- docker-compose.yml        Orquestacion de contenedores
|-- README.md                 Descripcion del proyecto y de los modulos
|-- config.php                Entorno, conexion PDO y ajustes de PHP
|-- conexion.php              Enlace PDO y preparacion del esquema
|-- auth.php                  Sesion, roles, matriz de permisos y CSRF
|-- correo.php                Cliente SMTP y avisos
|-- taller_disponibilidad.php Calculo de franjas y capacidad
|-- funciones_pago.php        Cobros, estado de cobro y facturas
|-- lang_setup.php            Configuracion del idioma de la sesion
|-- lang/es.php, lang/en.php  Textos de la interfaz
|-- index.php                 Portada publica
|-- login.php, logout.php     Acceso y salida
|-- registro.php              Alta de clientes
|-- panel_cliente.php         Panel del cliente
|-- dashboard_gestor.php      Panel del gestor
|-- dashboard_admin.php       Panel del administrador
|-- reservas.php              Turnos y vehiculos del cliente
|-- procesar_reserva.php      Confirmacion de la reserva
|-- cancelar_reserva.php      Cancelacion del turno
|-- perfil.php                Datos del usuario
|-- eliminar_cuenta.php       Baja de cuenta
|-- pagos_cliente.php         Pagos del cliente
|-- presupuestos.php          Presupuestos del cliente
|-- historial_cliente.php     Historial de servicios
|-- factura.php               Factura electronica
|-- taller.php                Vista publica de una sede
|-- disponibilidad.php        Franjas libres en vivo
|-- agenda_gestor.php         Agenda diaria del gestor
|-- detalle_reserva.php       Detalle de un turno
|-- administrar_reservas.php  Gestion de reservas
|-- administrar_pagos.php     Gestion de cobros
|-- administrar_presupuestos.php Gestion de presupuestos
|-- administrar_usuarios.php  Gestion de usuarios
|-- administrar_talleres.php  Gestion de sedes
|-- agregar_taller.php        Alta de sede
|-- editar_taller.php         Edicion de sede
|-- administrar_servicios.php Gestion de servicios
|-- permisos.php              Matriz de permisos
|-- datos_pago.php            Datos bancarios
|-- notificaciones.php        Avisos al cliente
|-- error_403.php             Acceso denegado
|-- css/                      estilos.css, tema-tgs.css, tgs-panel.css
|-- db/                       schema.sql, seed.sql, migracion.sql, vistas.sql
|-- scripts/                  _entorno.sh, instalar.sh, respaldar.sh,
                              restaurar.sh, caida.sh, estado.sh, operador.sh,
                              crear_roles_bd.sh
|-- docs/                     Documentacion tecnica, guia de defensa,
                              entregables, diagramas y el documento del
                              proyecto en PDF
```

Anexo C — Checklist normativo y pendientes

Verificación del cumplimiento de los requisitos administrativos del reglamento, con el estado real al cierre de este documento.

Exigencia	Estado	Observación
Grupo de 3 a 4 integrantes	Atender	El grupo tiene 5 integrantes: se debe solicitar la excepción por escrito al cuerpo docente y dejar constancia de la respuesta
Acta de integración oficial	Pendiente de firma	Debe estar impresa y firmada por los cinco, con los datos de cada uno y la designación del coordinador de proyecto y su suplente
Coordinador de proyecto titular	Pendiente de definir por escrito	Debe quedar establecido quién coordina y quién lo sustituye
Reglamento interno del grupo	Pendiente	Debe incluir los criterios de exclusión y las responsabilidades
Actas de reunión	Pendiente de consolidar	Las reuniones de seguimiento del proyecto deben documentarse con fecha, asistentes y acuerdos
CV de los cinco integrantes	Pendiente	Solo se cuenta con el CV de Santiago Figueroa
Entrega de la máquina virtual .OVA	Pendiente	Exportar la máquina con sus scripts y documentar el usuario y la clave del administrador
Protección del sistema con iptables	En curso	Dejar registrada la regla aplicada y verificarla en la defensa
Contenido de las otras asignaturas	Pendiente	Misión y visión de la empresa en inglés, análisis sociológico de resistencias, forma jurídica y trámites, estudio de mercado con matriz FODA, plan de marketing, y el modelo físico de la simulación con sus ecuaciones
Documentación bajo normas APA	**Cumplido**	Este documento: carátula, contenido, estructura de capítulos, figuras numeradas y referencias en APA 7º edición
Diagramas UML (6)	**Cumplido**	Capítulo 5: clases, actividades, secuencia, navegabilidad, despliegue y componentes
Documentación de base de datos	**Cumplido**	Capítulo 6: MER, diccionario de las 16 tablas, normalización, DDL, datos de prueba y migración
Plan de pruebas	**Cumplido**	Capítulo 9: plan con técnica y criterio de aprobación, y resultados de las 28 suites y los 20 casos
Control de versiones	**Cumplido**	Capítulo 10: repositorio GitHub con 18 commits descriptivos

> **Nota final.** Este documento se genera a partir del estado real del código y de la base de datos del proyecto: las capturas son del sistema funcionando en la máquina virtual, los diagramas se renderizan desde la notación versionada en el repositorio y las cifras (tablas, permisos, pruebas, commits) se toman del estado del proyecto al momento de la impresión.